

A PROJECT REPORT ON

MAAN SARATHI

AI-POWERED MENTAL WELLNESS PLATFORM WITH

HOLISTIC HEALING AND EMOTIONAL SUPPORT



Sinhgad Institutes

SUBMITTED TO

SAVITRIBAI PHULE PUNE UNIVERSITY, PUNE

IN PARTIAL FULFILMENT OF THE REQUIREMENTS FOR THE AWARD OF

BACHELOR OF ENGINEERING IN INFORMATION TECHNOLOGY

SUBMITTED BY

Student Name	Examination Number
Mr. Aditya Shirsat	B400430407
Mr. Atharva Lole	B400430382
Ms. Neha Kamble	B400430371
Mr. Sourabh Shirkande	B400430406

UNDER THE GUIDANCE OF

PROF. R. R. YADAV

DEPARTMENT OF INFORMATION TECHNOLOGY

STES'S SINHGAD ACADEMY OF ENGINEERING

KONDHWA (BK.), PUNE - 411048

ACADEMIC YEAR 2025-26

CERTIFICATE

This is to certify that the project report entitled “MaanSarathi: AI-Powered Mental Wellness Platform with Holistic Healing and Emotional Support” submitted by Mr. Aditya Shirsat, Mr. Atharva Lole, Ms. Neha Kamble, and Mr. Sourabh Shirkande is a bonafide record of work carried out under the guidance of Prof. M. S. Kale. The work is submitted in partial fulfilment of the requirements for the award of the Bachelor of Engineering degree in Information Technology of Savitribai Phule Pune University during the academic year 2025-26.

Project Guide / Internal Examiner	Head of Department
Prof. S.S.YADAV Signature: _____	Dr. S. S. Kulkarni Signature: _____

External Examiner	Principal
Name: _____ Signature: _____	Dr. S.S.Kulkarni Signature: _____

Place: Pune

Date: ____ / ____ / 2026

DECLARATION

We hereby declare that this project report is an original record of the work carried out by us under the guidance of Prof. M. S. Kale. The material presented in this report has not been submitted elsewhere for the award of any degree or diploma. External ideas, research findings, frameworks, libraries, and technical documentation used during development have been appropriately acknowledged.

Project Group Member	Signature
Mr. Aditya Shirsat	_____
Mr. Atharva Lole	
Ms. Neha Kamble	_____
Mr. Sourabh Shirkande	



Date:-

To,
Dr. M. S. Rohokale.,
Principal,
Sinhgad Academy Of Engineering, Pune.

Subject: - Regarding sponsorship of Projects.

Dear Sir,

With reference your Sponsorship request, this is to inform you that our Organization can Sponsor One Project. We have Shortlist projects as per our previous discussion for the Sponsorship & we are here with attaching the project details of projects.

Sr.No	Title of Project	Domain of the Project	Name of Students
01	Manasarthi AI Powered Mental wellness platform with holistic healing and emotional support.	Healthtech	Atharva Iole
			Aditya Shirsat
			Sourabha Shirkande
			Neha Suresh Kamble



Gosavi.V.S

Yours Sincerely,
Vishal S Gosavi,
Director

973 003 0609 / 935 926 6248

vishalstepup2025@gmail.com

www.stepupsolutions.in

A301, Mayuri Infinity, Behind Mayuri Vajankata,
Undri, Pune

ACKNOWLEDGEMENT

would like to thank my Guide, Prof. R.R.Yadav whose supervision and valuable discussion has helped us tremendously to complete our project on” Manasarthi AI Powered Mental wellness platform with holistic healing and emotional support.”. His guidance proved to be valuable to overcome all the hurdles in the fulfilment of this project.

I express my sincere appreciation towards the efforts taken by our project co-ordinator Dr.

S. L. Bangare whose sincere guidance and leadership have helped us achieve the milestones set periodically.

I am grateful to Dr. S. S. Kulkarni, Head of Information Technology Department for providing all the necessary facilities and help. I also thank all the teaching and non-teaching staff of Information Technology Department, for their direct and indirect help in the completion of the project.

We express our sincere gratitude towards co-operative department who have provided us with valuable assistance and requirements for the presentation.

Mr. Aditya Shirsat

Mr. Atharva Lole

Miss. Neha Kamble

Mr. Sourabha Shirkande

ABSTRACT

MaanSarathi is a full-stack, AI-assisted mental wellness platform designed to improve access to structured self-care, emotional reflection, and professional oversight. The system combines user onboarding, standardized mental-health assessments, mood and sleep tracking, journaling, gratitude and habit features, personalized wellness plans, multimedia practices, progress analytics, a crisis-aware AI companion, an administrator portal, and a therapist portal.

The web application is implemented using React 18, Vite, TypeScript, Tailwind CSS, Radix UI, Zustand, TanStack React Query, and i18next. The backend uses Express.js and TypeScript with Prisma ORM, SQLite for local development, and PostgreSQL for production. Authentication is provided through JWT and Google OAuth. A unified AI provider service supports Gemini, OpenAI, Anthropic, Hugging Face, NVIDIA, and Ollama with priority-based fallback and cooldown handling.

Safety is treated as a core requirement. Crisis detection combines keyword patterns, AI-assisted risk classification, assessment history, mood trajectory, and engagement context. The platform provides safety plans, emergency resources, support-ticket workflows, data export, privacy controls, and account deletion. The completed implementation demonstrates that a modular architecture can combine personalization, data-driven wellbeing insights, and role-based oversight while retaining clear non-clinical boundaries.

Keywords: Artificial Intelligence, Mental Wellness, Conversational AI, Crisis Detection, Cognitive Behavioural Therapy, Clinical Assessment, React, Express, Prisma, Data Privacy.

TABLE OF CONTENTS

INTRODUCTION.....	11
1.1 Background.....	11
1.2 Problem Statement.....	11
1.3 Aim	11
1.4 Objectives	11
1.5 Scope	11
1.6 Project Contributions	12
1.7 Report Organization	12
LITERATURE SURVEY	13
2.1 Review of Existing Work	13
2.2 Gap Analysis.....	13
2.3 Proposed Resolution	14
SOFTWARE REQUIREMENTS SPECIFICATION	15
3.1 Stakeholders and User Roles	15
3.2 Functional Requirements	15
3.3 Non-Functional Requirements.....	16
3.4 Hardware and Software Requirements	16
3.5 Constraints and Assumptions	17
3.6 Feasibility Study	17
SYSTEM ANALYSIS AND DESIGN	18
4.1 Architecture Overview	18
4.2 Component Architecture.....	18
4.3 Data Flow Design	19
4.4 Use-Case Design.....	21
4.5 Database Design	22
4.6 Sequence Design.....	24
4.7 Activity and State Design	26
PROJECT PLANNING AND MANAGEMENT	29

5.1 Development Methodology	29
5.2 Work Breakdown.....	29
5.3 Schedule.....	30
5.4 Risk Management	30
IMPLEMENTATION	31
6.1 Technology Stack	31
6.2 Frontend Implementation	31
6.3 Backend API Implementation	32
6.4 Authentication and Authorization	32
6.5 Database Implementation	33
6.6 Multi-Provider AI Service	33
6.7 Assessment and Insight Processing	33
6.8 Personalization and Recommendations	33
SECURITY, PRIVACY, SAFETY AND ETHICS	34
7.1 Security Controls	34
7.2 Privacy and Data Control	34
7.3 Crisis Detection	34
7.4 Ethical Boundaries.....	34
TESTING AND VALIDATION.....	36
8.1 Testing Strategy	36
8.2 Representative Test Cases	36
8.3 Validation Considerations	37
RESULTS AND DISCUSSION	38
9.1 Implemented User Experience.....	38
9.2 Result Summary	40
9.3 Discussion.....	41
DEPLOYMENT, OPERATION AND MAINTENANCE	42
10.1 Local Setup	42
10.2 Production Architecture.....	42
10.3 Health and Monitoring.....	42
10.4 Backup and Recovery	42

CONCLUSION AND FUTURE SCOPE	43
11.1 Conclusion	43
11.2 Limitations	43
11.3 Future Scope	43
REFERENCES	44
APPENDIX A	45
PROJECT DIRECTORY STRUCTURE	45
APPENDIX B	46
ENVIRONMENT CONFIGURATION REFERENCE	46
APPENDIX C	47
IMPORTANT OPERATING COMMANDS	47

LIST OF FIGURES

Figure 4.1 System Architecture Diagram

Figure 4.2 Component Architecture Diagram

Figure 4.3 Data Flow Diagram - System Context

Figure 4.4 Data Flow Diagram - Levels 0, 1 and 2

Figure 4.5 Overall Use-Case Diagram

Figure 4.6 User Module Use-Case Diagram

Figure 4.7 Admin Module Use-Case Diagram

Figure 4.8 Therapist Module Use-Case Diagram

Figure 4.9 Entity-Relationship Diagram

Figure 4.10 Class Diagram

Figure 4.11 Assessment Sequence Diagram

Figure 4.12 Chat and AI Sequence Diagram

Figure 4.13 AI Conversation ChatFlow

Figure 4.14 End-to-End User Activity Diagram

Figure 4.15 Crisis Detection Activity Diagram

Figure 4.16 Chat/User State Machine

Figure 5.1 Project Timeline - Gantt Chart

Figure 9.1 User Authentication Screen

Figure 9.2 Public Landing Page

Figure 9.3 AI Companion Interface

Figure 9.4 Admin Dashboard

LIST OF TABLES

Table 2.1 Literature Survey Summary

Table 3.1 Functional Requirements

Table 3.2 Non-Functional Requirements

Table 3.3 Hardware and Software Requirements

Table 4.1 Major System Components

Table 4.2 Principal Database Entities

Table 5.1 Project Plan and Milestones

Table 6.1 Technology Stack

Table 6.2 Core API Groups

Table 8.1 Testing Strategy

Table 8.2 Representative Test Cases

Table 9.1 Result Summary

ABBREVIATIONS

Abbreviation	Meaning
AI	Artificial Intelligence
API	Application Programming Interface
CBT	Cognitive Behavioural Therapy
CSRF	Cross-Site Request Forgery
DFD	Data Flow Diagram
JWT	JSON Web Token
LLM	Large Language Model
NFR	Non-Functional Requirement
ORM	Object-Relational Mapping
PWA	Progressive Web Application
SRS	Software Requirements Specification
UML	Unified Modelling Language

CHAPTER 1

INTRODUCTION

CHAPTER 1

INTRODUCTION

1.1 Background

Mental wellbeing is influenced by academic pressure, employment uncertainty, social isolation, relationship stress, sleep quality, physical health, and access to support. Many people delay seeking professional help because of stigma, cost, limited availability, or uncertainty about where to begin. Digital wellness tools can provide private, immediate, and structured support, but they must be designed with safety, transparency, and non-clinical boundaries.

1.2 Problem Statement

Existing wellness applications often provide isolated features such as mood tracking, meditation, or chat. Users must move between multiple applications, while professionals and administrators receive limited contextual information. There is a need for an integrated platform that combines assessments, reflective tracking, AI-assisted conversation, personalized resources, progress visualization, crisis-aware assistance, and role-based oversight without presenting itself as a replacement for licensed medical care.

1.3 Aim

To design and implement a secure, modular, AI-powered mental wellness platform that supports self-reflection, early awareness, personalized wellbeing practices, crisis-aware conversational assistance, and professional oversight.

1.4 Objectives

- Provide secure user registration, authentication, onboarding, and profile management.
- Offer standardized assessments and store historical results for trend analysis.
- Provide mood, sleep, journal, habit, intention, gratitude, and micro-check-in tracking.
- Implement a context-aware AI companion with provider fallback and conversation continuity.
- Detect crisis indicators and surface grounding exercises, safety plans, and emergency resources.
- Generate personalized wellness plans and content recommendations.
- Provide administrative tools for content, assessment, user, therapist, support, and safety management.
- Provide a therapist portal for bookings, client context, and session notes.
- Support privacy preferences, data export, and account deletion.

- Maintain a deployable architecture for local development and cloud production.

1.5 Scope

The implemented scope covers a responsive web application and backend API. The repository currently contains active frontend and backend workspaces. The platform is intended for wellbeing support, reflection, education, and connection to resources. It does not diagnose disorders, prescribe medication, or replace emergency services or professional treatment.

1.6 Project Contributions

- Unified data model connecting daily tracking, assessments, conversations, plans, content engagement, and professional workflows.
- Multi-provider AI integration with availability checks, failure cooldowns, and fallback.
- Multi-layer crisis detection using conversation content, assessment history, mood trajectory, and engagement.
- Separate user, administrator, and therapist experiences within one platform.
- Local SQLite and production PostgreSQL support through automated Prisma setup.
- Privacy and safety features integrated into application architecture instead of added as isolated screens.

1.7 Report Organization

Chapter 2 reviews related work. Chapter 3 defines requirements. Chapter 4 presents system design and architecture. Chapter 5 explains planning. Chapter 6 describes implementation. Chapter 7 covers security, privacy, safety, and ethics. Chapter 8 presents testing. Chapter 9 discusses results. Chapter 10 describes deployment and maintenance. Chapter 11 concludes the work and identifies future scope.

CHAPTER 2

LITERATURE SURVEY

CHAPTER 2

LITERATURE SURVEY

2.1 Review of Existing Work

Research on conversational agents indicates that automated support can improve accessibility and engagement when it uses structured psychological techniques and clearly communicates limitations. Woebot demonstrated the feasibility of delivering CBT-oriented conversations to young adults. Recent reviews identify growing use of large language models in mental-health support, while consistently warning about hallucinations, privacy, cultural limitations, crisis response, and the need for human oversight.

Studies on AI and positive mental health highlight opportunities in emotional regulation, psychoeducation, screening, and personalization. Indian and culturally diverse contexts require language flexibility, locally relevant resources, sensitivity to family and spiritual practices, and careful handling of stigma. These observations informed MaanSarathi's hybrid wellbeing approach, multilingual interface, configurable content system, and therapist/admin roles.

Table 2.1 Literature Survey Summary

Study	Contribution	Identified Gap
Fitzpatrick et al. (2017)	Automated CBT conversational agent evaluated with young adults.	Limited broader personalization and integrated longitudinal tracking.
Thakkar et al. (2024)	Narrative review of AI in positive mental health.	Ethics, cultural sensitivity, and personalization remain concerns.
Casu et al. (2024)	Scoping review of chatbot effectiveness and feasibility.	Long-term safety and real-world integration require further evidence.
Marade et al. (2025)	AI platform combining mental, emotional, and spiritual wellbeing.	Limited large-scale validation and operational detail.
Halder (2025)	Challenges of mental-health chatbots in India.	Infrastructure, language, access, and clinical governance challenges.
Méndez et al. (2024)	Deployment of a mental-health chatbot in higher education.	Institutional deployment needs monitoring and escalation workflows.

2.2 Gap Analysis

- Many applications separate chat, assessments, tracking, and content into unrelated products.
- Single-provider AI systems are vulnerable to service interruption and quota failures.
- Safety is frequently implemented as simple keyword matching without longitudinal context.

- Administrative content governance and therapist workflows are often absent.
- Users receive limited control over consent, data export, and deletion.
- Cultural and multilingual personalization remains limited.

2.3 Proposed Resolution

MaanSarathi addresses these gaps through a modular platform that joins assessment data, daily wellbeing records, AI conversation memory, personalized recommendations, help and safety resources, and professional portals. The architecture separates UI, API, service, provider, and persistence responsibilities so that individual modules can evolve without redesigning the whole system.

CHAPTER 3

SOFTWARE REQUIREMENTS SPECIFICATION

CHAPTER 3

SOFTWARE REQUIREMENTS SPECIFICATION

3.1 Stakeholders and User Roles

Role	Responsibilities
End User	Completes onboarding, assessments, tracking, chat, practices, journal, privacy controls, support requests, and progress review.
Administrator	Manages users, therapists, assessments, content, practices, media, FAQ, support, safety resources, analytics, and activity logs.
Therapist	Uses the therapist portal for client context, bookings, calendar, and session notes.
System Operator	Configures environment variables, database, AI providers, deployment, logging, backups, and monitoring.

3.2 Functional Requirements

Table 3.1 Functional Requirements

ID	Module	Requirement
FR-01	Authentication	Email/password registration and login, Google OAuth, password setup, token validation, logout.
FR-02	Onboarding	Capture demographic, language, regional, consent, emergency-contact, and wellbeing preferences.
FR-03	Assessments	Run individual and combined assessments, calculate scores, store history, and generate insights.
FR-04	AI Companion	Provide contextual chat, conversation history, memory, continuity, feedback, and export.
FR-05	Crisis Safety	Detect risk indicators and provide immediate resources, safety plans, grounding, and event tracking.
FR-06	Daily Tracking	Record mood, micro-check-ins, sleep, journal, habits, intentions, and gratitude.
FR-07	Plans and Content	Generate plans and recommend articles, audio, video, breathing, yoga, CBT, and mindfulness practices.
FR-08	Progress	Show longitudinal trends, wellness snapshots, engagement, and assessment changes.
FR-09	Support	Provide FAQ, support tickets, crisis resources, and help/safety workflows.
FR-10	Privacy	Manage consent, clinician sharing, analytics preferences, export, and account deletion.

ID	Module	Requirement
FR-11	Administration	Manage platform data, content, users, assessments, media, safety, analytics, and diagnostics.
FR-12	Therapist Portal	Authenticate therapists, manage bookings/calendar, view assigned client context, and maintain notes.

3.3 Non-Functional Requirements

Table 3.2 Non-Functional Requirements

Category	Requirement
Security	JWT/OAuth authentication, password hashing, CSRF protection, sanitization, role checks, secret isolation, and production HTTPS.
Safety	Non-clinical disclaimers, crisis-aware handling, emergency resources, controlled AI prompts, and auditable crisis events.
Performance	Responsive UI, cached client queries, optimized database indexes, request timeout, compression, and provider availability caching.
Reliability	AI fallback, readiness checks, graceful shutdown, structured logging, and local fallback behavior.
Usability	Responsive web interface, accessible primitives, loading/error states, clear navigation, and multilingual UI.
Maintainability	TypeScript, feature-based frontend, route/controller/service backend separation, central configuration, and automated scripts.
Scalability	PostgreSQL production database, stateless API patterns, modular services, and independent frontend/backend deployment.
Privacy	Consent preferences, restricted sharing, data export, account deletion, and minimal exposure of sensitive records.

3.4 Hardware and Software Requirements

Table 3.3 Hardware and Software Requirements

Component	Specification
Developer System	Modern multi-core processor, minimum 8 GB RAM, 10 GB free storage.
User Device	Desktop, tablet, or mobile device with a modern browser and internet access.
Runtime	Node.js 18+, npm 8+.
Frontend	React 18, Vite 5, TypeScript, Tailwind CSS, Radix UI.
Backend	Express.js, TypeScript, Prisma ORM.
Database	SQLite for local development; PostgreSQL for production.

Component	Specification
AI	At least one configured provider API key or local Ollama fallback.
Deployment	Vercel-compatible frontend and Render-compatible backend configuration.

3.5 Constraints and Assumptions

- AI output quality depends on provider availability, model behavior, and configured API limits.
- The platform is a wellbeing aid and not a diagnostic or emergency medical system.
- Production use requires HTTPS, secure secrets, approved privacy policy, and professional clinical governance.
- Assessment interpretations must remain aligned with validated scoring guidance.
- Internet connectivity is required for cloud AI and production API access.

3.6 Feasibility Study

Technical feasibility is supported by mature open-source frameworks and the modular codebase. Economic feasibility is supported by free development tools, local SQLite, and configurable cloud/AI providers. Operational feasibility is supported by role-based workflows and a responsive interface. Ethical feasibility requires explicit consent, privacy controls, transparent limitations, and escalation to human or emergency support when risk is detected.

CHAPTER 4

SYSTEM ANALYSIS AND DESIGN

CHAPTER 4

SYSTEM ANALYSIS AND DESIGN

4.1 Architecture Overview

MaanSarathi follows a layered architecture consisting of client, business, AI, and data layers. The React-based web and mobile applications provide user interaction and navigation. The Node.js and Express.js backend manages APIs, authentication, validation, and business logic. AI services handle chatbot conversations, recommendations, NLP processing, and crisis detection. PostgreSQL, Redis, and object storage are used for data persistence, caching, and file management. External services such as AI providers, authentication systems, and notification services are integrated through dedicated service layers, ensuring scalability, security, and maintainability.

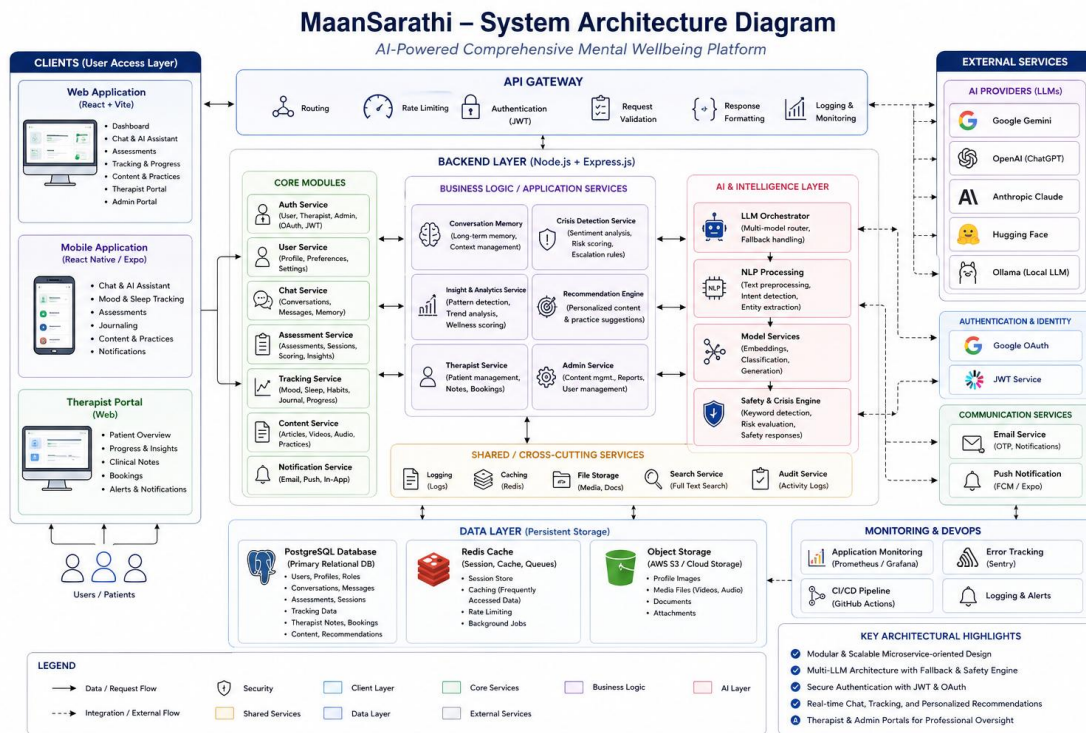


Figure 4.1 System Architecture Diagram

4.2 Component Architecture

Table 4.1 Major System Components

Layer	Major Components	Responsibility
Presentation	User app, admin portal, therapist portal, UI primitives	Interaction, navigation, visualization, accessibility, localization.
Client State	Zustand, React Context, TanStack React Query	Authentication state, app preferences, notifications, server cache.
API	Express routes and controllers	HTTP contracts, validation, authentication, response handling.
Domain Services	Chat, assessments, memory, recommendations, crisis, analytics	Business rules and cross-module orchestration.
AI Provider Layer	Gemini, OpenAI, Anthropic, Hugging Face, NVIDIA, Ollama	Provider abstraction, availability, fallback, cooldown, usage logging.
Persistence	Prisma Client, SQLite/PostgreSQL	Relational data, indexes, constraints, transactions.
Operations	Health checks, Pino logs, environment validation, graceful shutdown	Reliability, diagnostics, configuration, deployment readiness.

Component Diagram – MaanSarathi System Architecture

High-level view of major components and their interactions

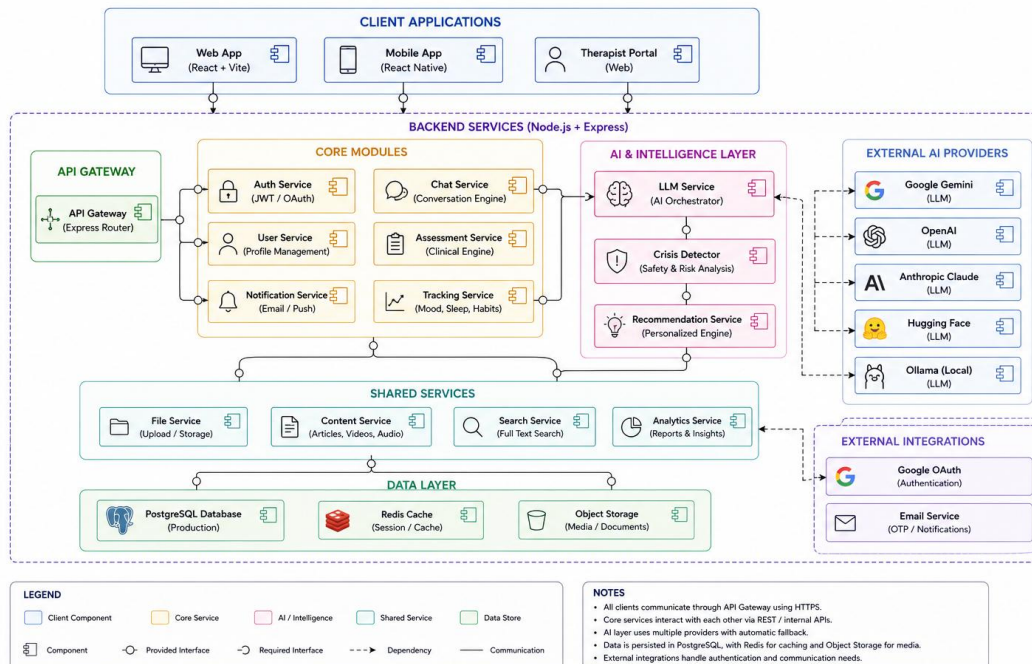


Figure 4.2 Component Architecture Diagram

4.3 Data Flow Design

Users interact with the web client, which sends authenticated requests to the API. Middleware validates tokens, CSRF state, input content, rate limits, and request timeouts. Routes delegate to controllers and services. Services read or modify Prisma entities and may call AI providers or email/media services. Responses are returned as structured JSON and cached by the client where appropriate.

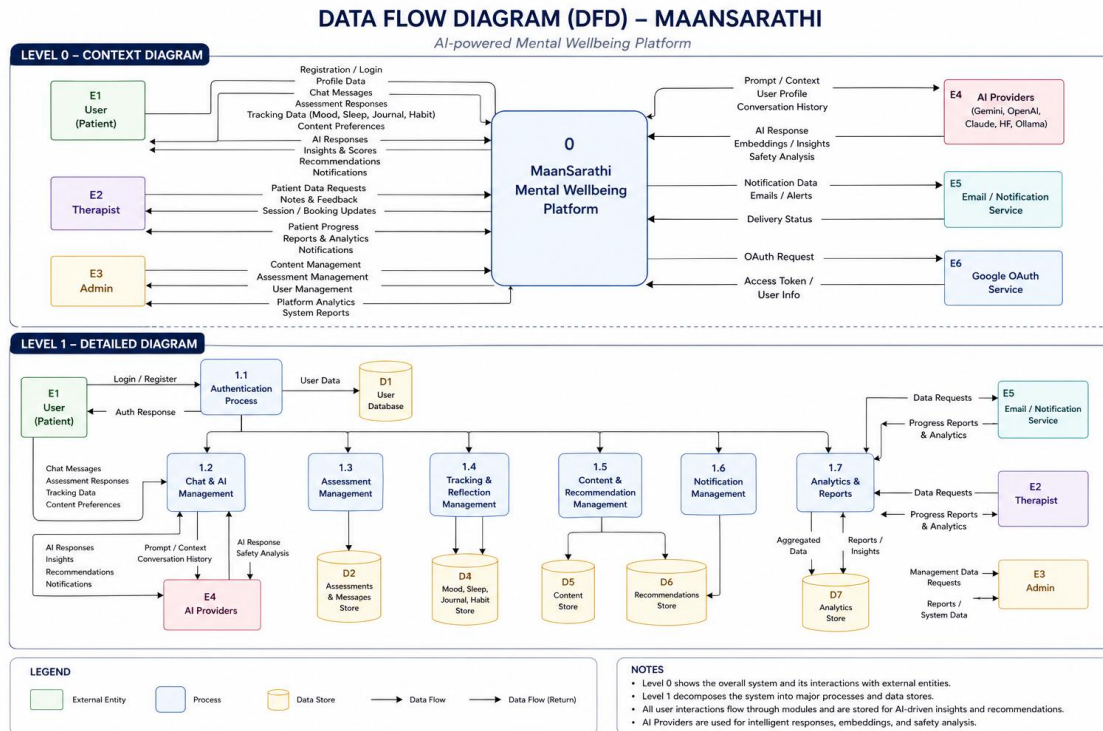


Figure 4.3 Data Flow Diagram - System Context

MAANSARATHI – DATA FLOW DIAGRAMS (DFD)

AI-Powered Mental Wellbeing Platform

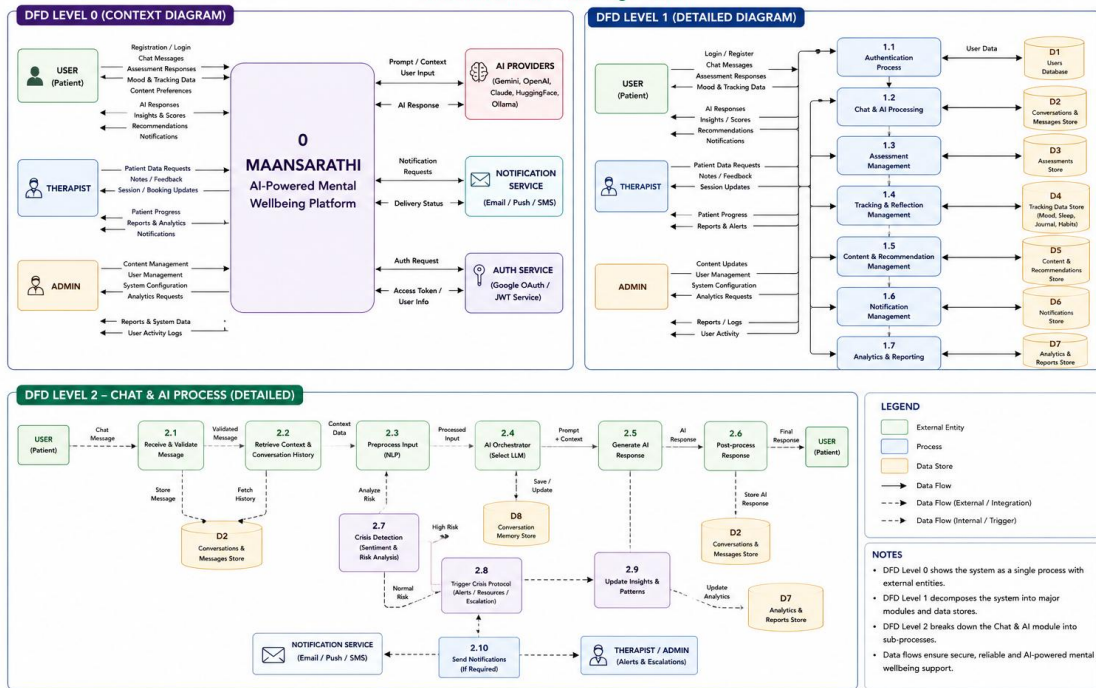


Figure 4.4 Data Flow Diagram - Levels 0, 1 and 2

4.4 Use-Case Design

Overall System Use Case Diagram – MaanSarathi

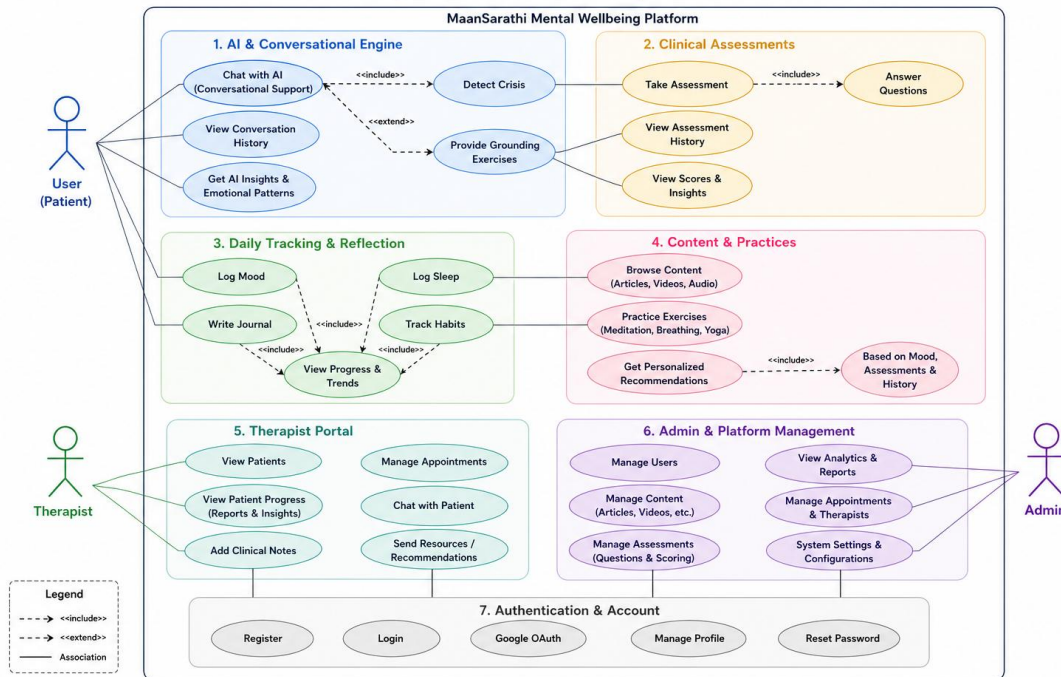


Figure 4.5 Overall Use-Case Diagram

USER MODULE USE CASE DIAGRAM - MAANSARATHI PLATFORM

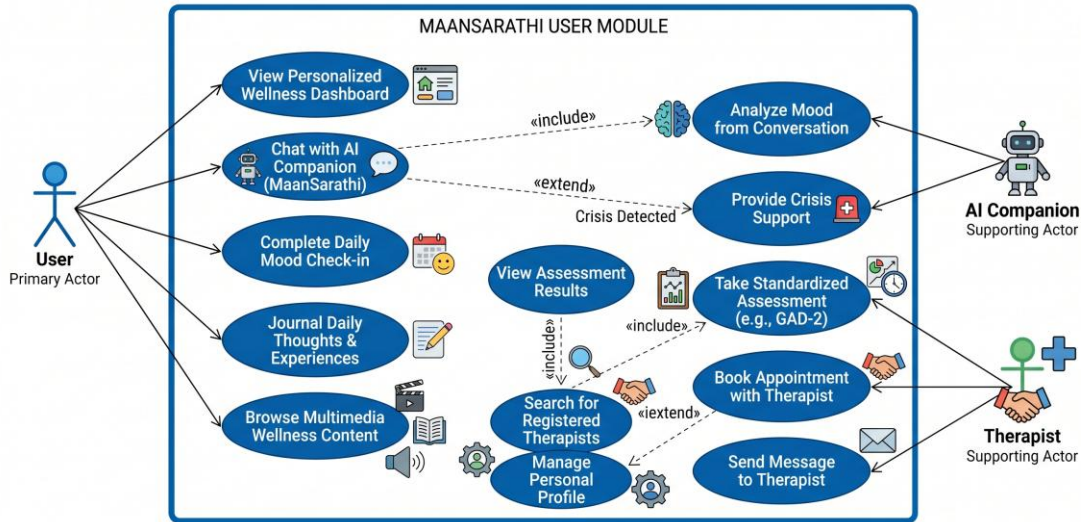


Figure 4.6 User Module Use-Case Diagram

ADMIN MODULE USE CASE DIAGRAM - MAANSARATHI PLATFORM

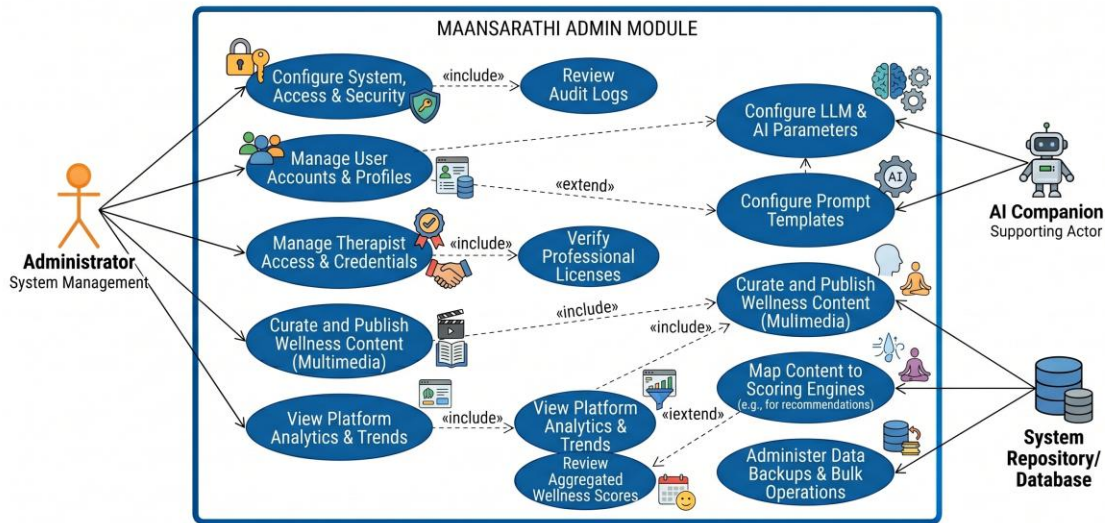


Figure 4.7 Admin Module Use-Case Diagram

THERAPIST MODULE USE CASE DIAGRAM - MAANSARATHI PLATFORM

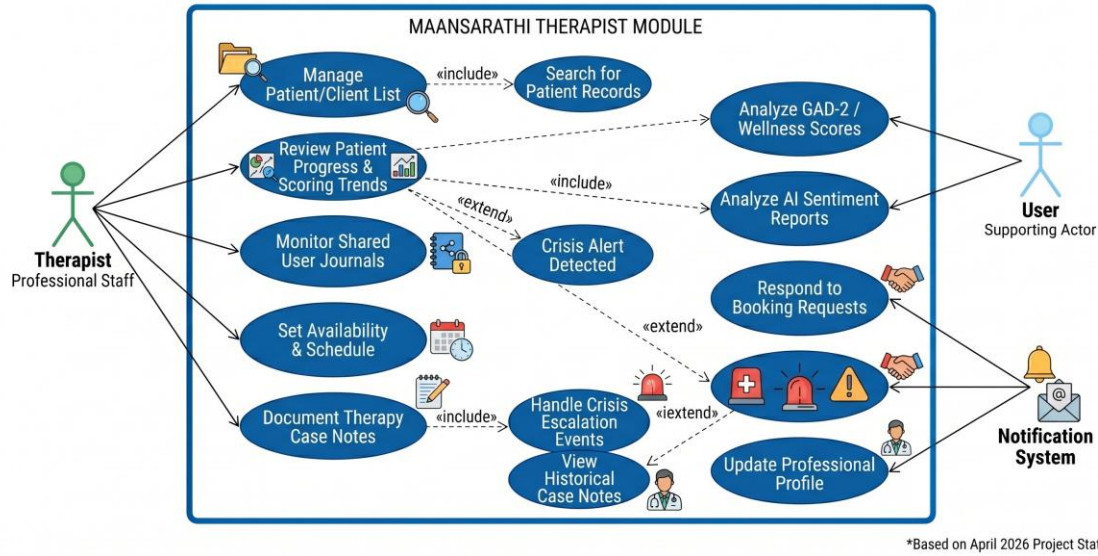


Figure 4.8 Therapist Module Use-Case Diagram

4.5 Database Design

The User entity is the central ownership boundary. Related records include assessment sessions/results, mood entries, check-ins, journals, intentions, gratitude, habits, sleep logs, plan modules, conversations, chat messages, memory, goals, progress tracking, content engagement, support tickets, safety plans, therapist bookings/notes, crisis events, sessions, and wellness snapshots. Cascading deletion and indexes support integrity and common query paths.

Table 4.2 Principal Database Entities

Entity Group	Representative Entities
Identity and Consent	User, privacy preferences, email verification, OAuth identifiers.
Assessment	AssessmentSession, AssessmentResult, AssessmentInsight.
Daily Wellbeing	MoodEntry, MicroCheckin, SleepLog, JournalEntry, DailyIntention, GratitudeEntry, UserHabit.
Conversation	Conversation, ChatMessage, ConversationMemory, ConversationGoal, ChatFeedback.
Plans and Content	UserPlanModule, Content, ContentEngagement, Practice.
Safety and Support	CrisisEvent, SafetyPlan, CrisisResource, SupportTicket, FAQ.
Professional	Therapist, TherapistBooking, TherapistNote.
Analytics	ProgressTracking, WellnessSnapshot, UserSession, DashboardInsights.

ER Diagram – MaanSarathi (Database Design)

AI-powered Mental Wellbeing Platform

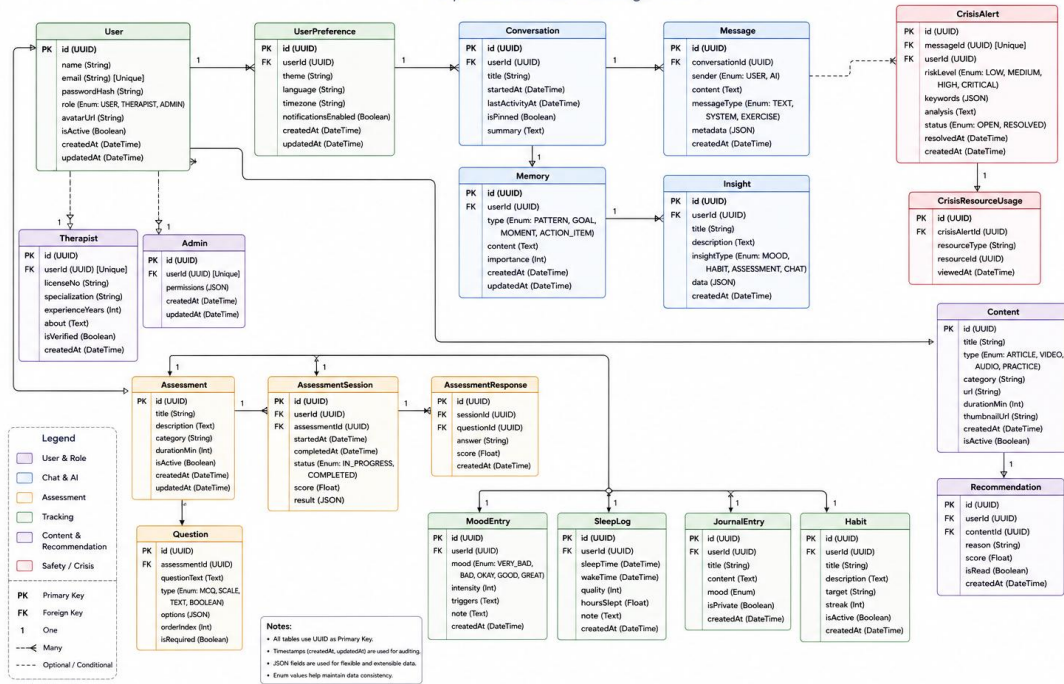


Figure 4.9 Entity-Relationship Diagram

Class Diagram – MaanSarathi (Main System Architecture)

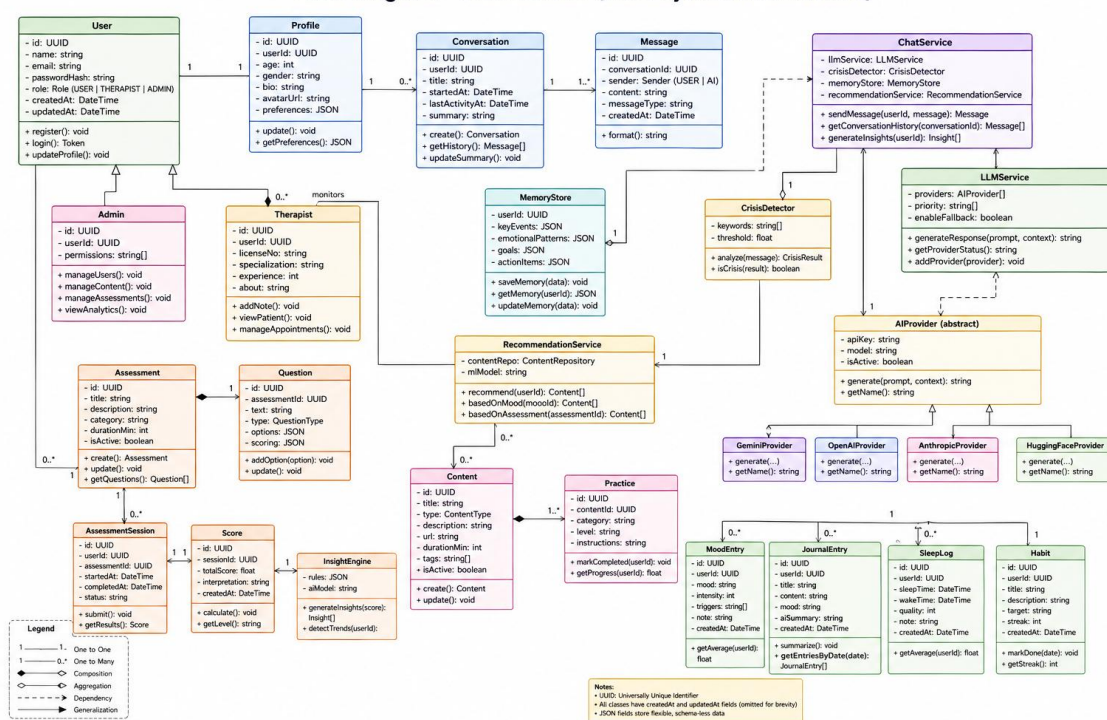


Figure 4.10 Class Diagram

4.6 Sequence Design

Assessment flow creates or resumes a session, records responses, applies assessment-specific scoring, stores results, and refreshes insights. Chat flow loads context and memory, evaluates safety, builds prompts, selects an available provider, stores the response, and updates continuity data.

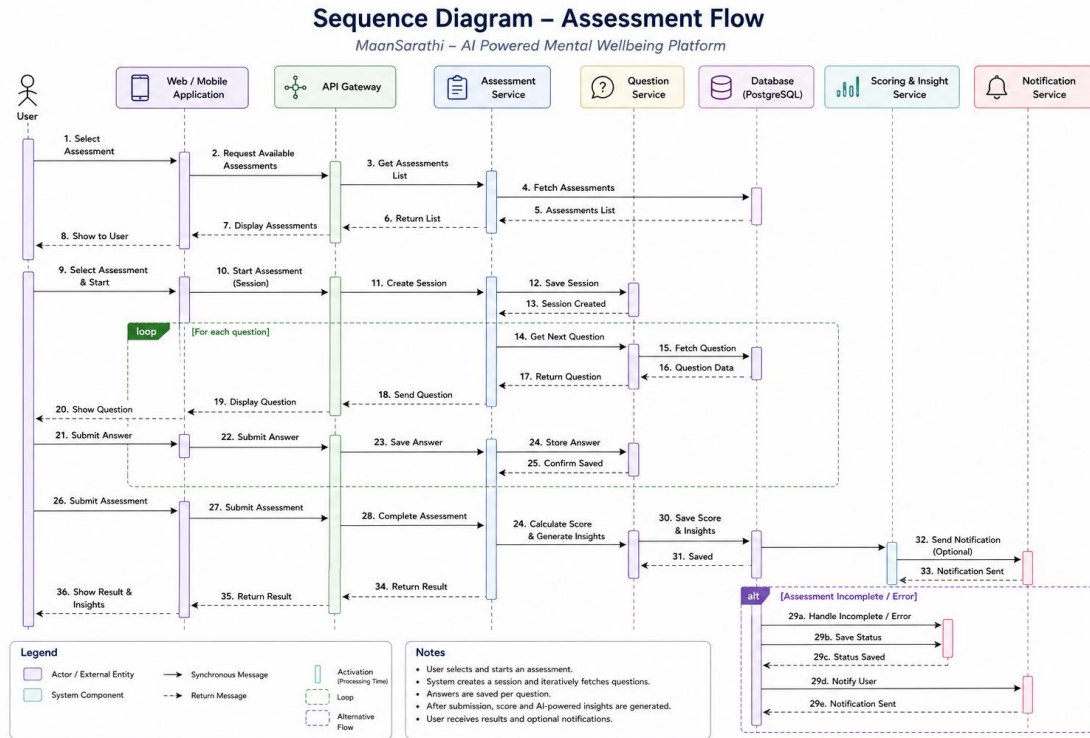


Figure 4.11 Assessment Sequence Diagram

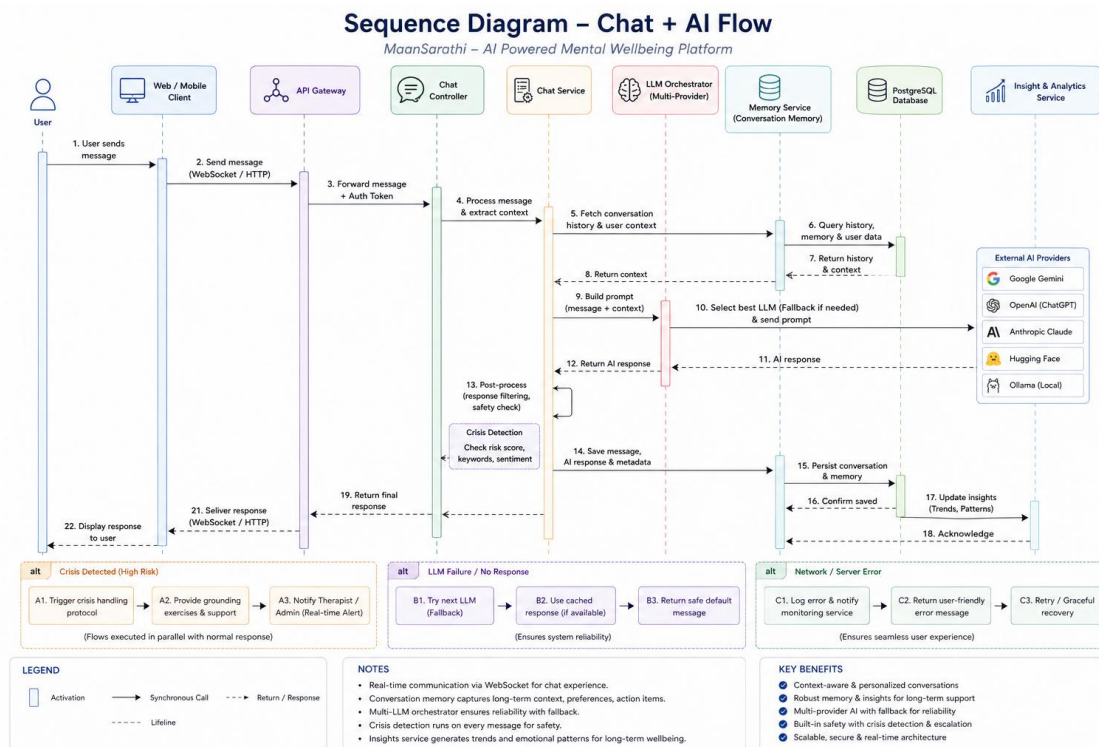


Figure 4.12 Chat and AI Sequence Diagram

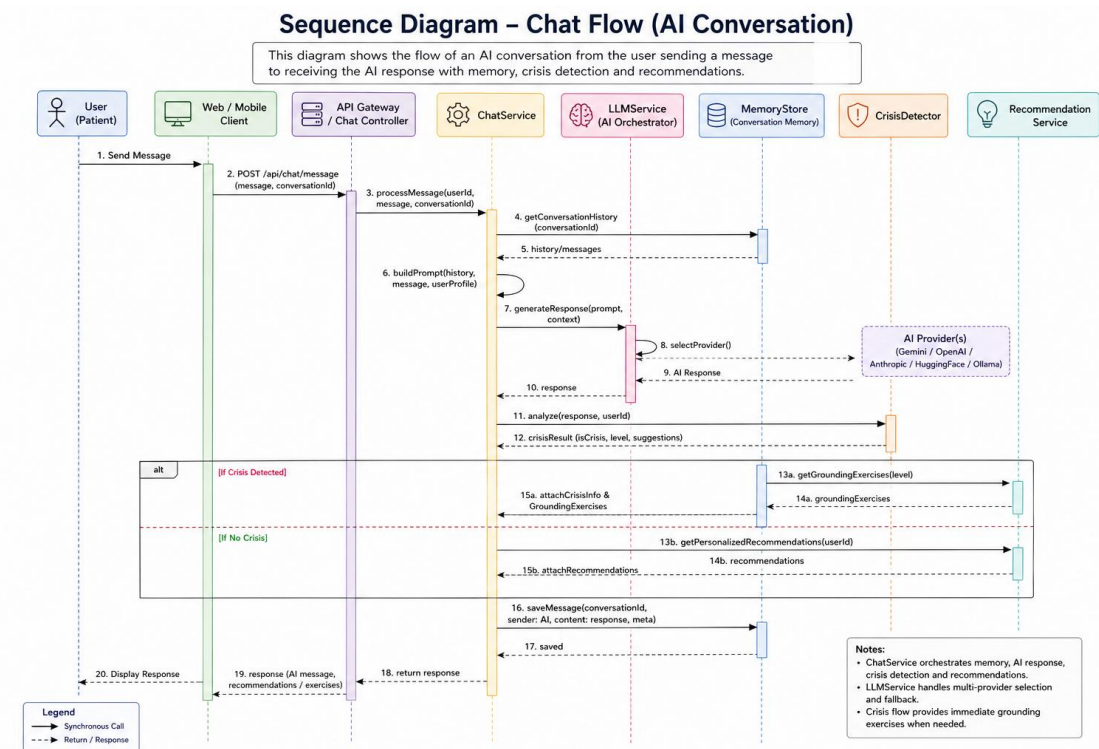


Figure 4.13 AI Conversation ChatFlow

4.7 Activity and State Design

Activity Diagram – User Journey (End-to-End Flow)

MaanSarathi Mental Wellbeing Platform

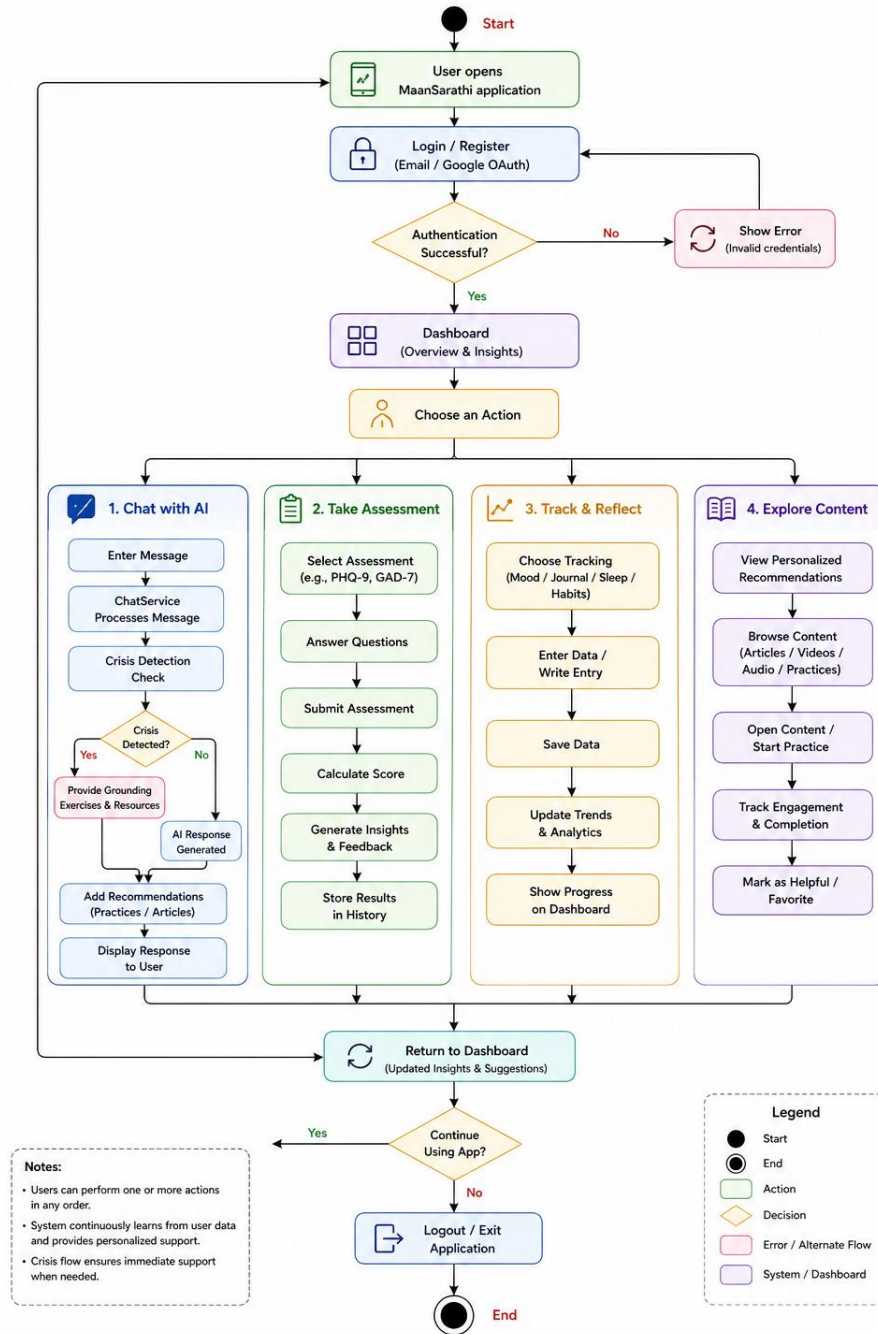


Figure 4.14 End-to-End User Activity Diagram

Activity Diagram – Crisis Detection Logic (Advanced AI Flow)

MaanSarathi – AI Powered Mental Wellbeing Platform

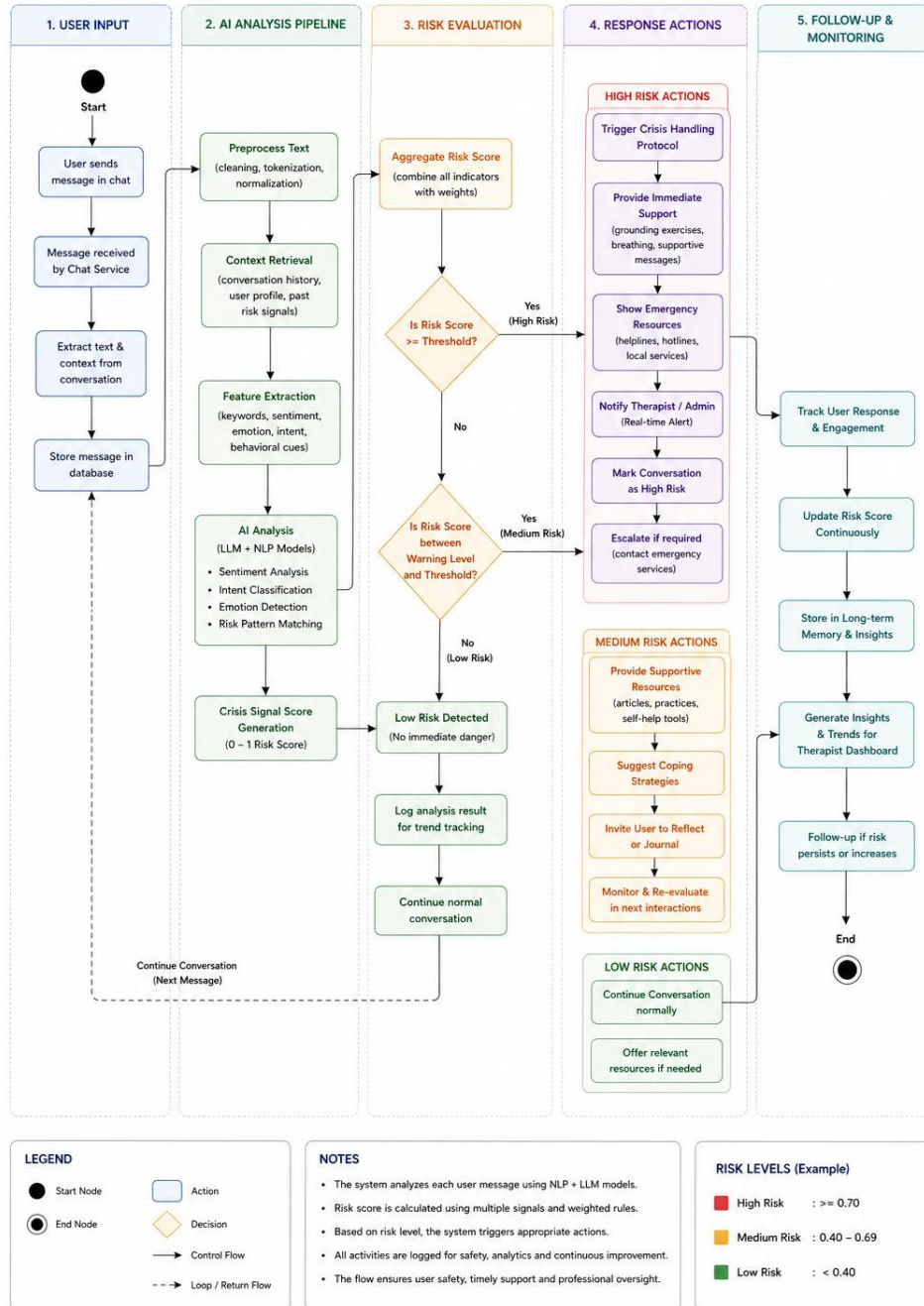


Figure 4.15 Crisis Detection Activity Diagram

State Machine Diagram – Chat / User State Lifecycle

MaanSarathi Mental Wellbeing Platform

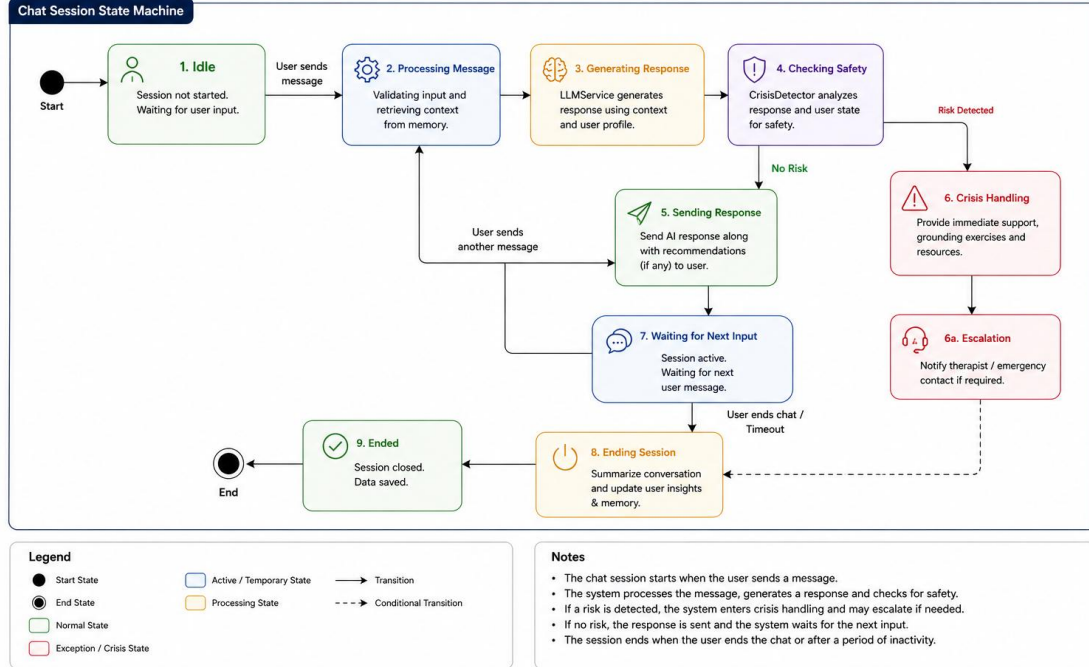


Figure 4.16 Chat/User State Machine

CHAPTER 5

PROJECT PLANNING AND MANAGEMENT

CHAPTER 5

PROJECT PLANNING AND MANAGEMENT

5.1 Development Methodology

The project followed an iterative and incremental approach. Core authentication and data models were established first, followed by assessments, dashboard features, AI chat, daily tracking, admin tools, therapist workflows, safety features, testing, and deployment configuration. Feedback from each iteration was used to refine API contracts and interface behavior.

5.2 Work Breakdown

Table 5.1 Project Plan and Milestones

Phase	Activities	Deliverables
Problem Definition	Domain study, user roles, safety boundaries, scope.	Problem statement and objectives.
Requirement Analysis	Functional/NFR analysis, module identification.	SRS and use cases.
System Design	Architecture, database, DFD, UML, API planning.	Design diagrams and schema.
Core Development	Frontend, backend, authentication, assessments, tracking.	Integrated web application.
AI and Safety	Provider layer, memory, crisis detection, fallback.	Crisis-aware AI companion.
Professional Modules	Admin and therapist workflows.	Role-based portals.
Verification	Type checks, unit tests, integration checks, UI review.	Test reports and fixes.
Deployment	Environment scripts, production config, documentation.	Deployable system and Black Book.

5.3 Schedule

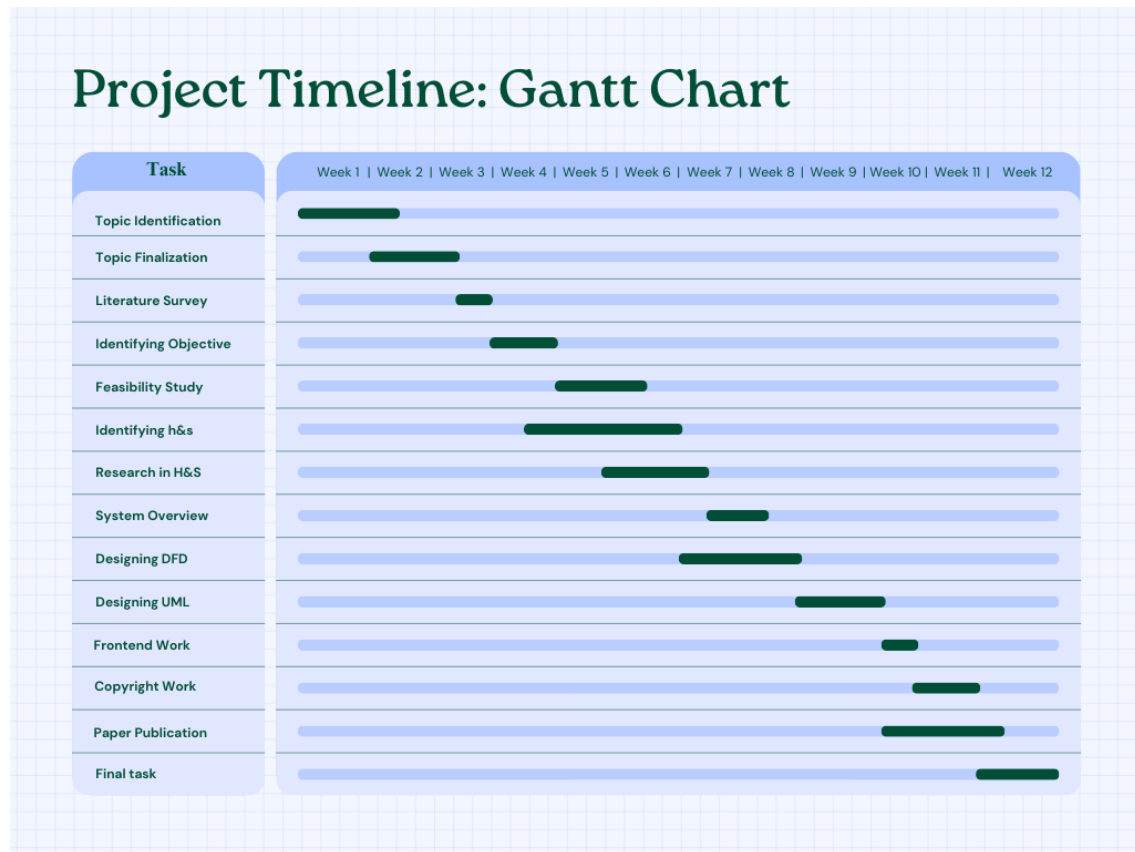


Figure 5.1 Project Timeline - Gantt Chart

5.4 Risk Management

Risk	Impact	Mitigation
AI provider failure	Chat interruption	Multiple providers, fallback, cooldown, local development fallback.
Unsafe AI response	User harm	Safety prompts, crisis detection, non-clinical boundaries, emergency resources.
Sensitive-data exposure	Privacy and trust loss	JWT/OAuth, access control, environment secrets, consent and export/delete controls.
Database inconsistency	Incorrect insights	Prisma relations, constraints, indexes, local seed scripts, migrations.
Scope growth	Delayed delivery	Feature-based milestones and prioritization of core workflows.
Deployment misconfiguration	Unavailable service	Environment doctor, validation, health/readiness endpoints.

CHAPTER 6

IMPLEMENTATION

CHAPTER 6

IMPLEMENTATION

6.1 Technology Stack

Table 6.1 Technology Stack

Area	Technology	Purpose
Frontend	React 18, Vite, TypeScript	Component-based responsive web application.
UI	Tailwind CSS, Radix UI, Lucide	Styling, accessible primitives, iconography.
State/Data	Zustand, Context, TanStack React Query	Local state and server-data caching.
Backend	Node.js, Express, TypeScript	REST API and middleware pipeline.
Database	Prisma, SQLite, PostgreSQL	Typed ORM and environment-specific storage.
Authentication	JWT, bcrypt, Passport, Google OAuth	Identity and protected API access.
AI	Gemini, OpenAI, Anthropic, Hugging Face, NVIDIA, Ollama	Configurable conversational intelligence.
Testing	Vitest, Supertest, Testing Library	Unit, integration, and frontend verification.
Operations	Pino, Helmet, compression, rate limiting	Logging, security headers, performance, abuse control.

6.2 Frontend Implementation

The frontend uses a feature-based structure. App.tsx coordinates authentication, onboarding, assessments, and page rendering. Path mapping is centralized in appRouting.ts. API operations are grouped in service modules, while Zustand stores maintain authentication, application, and notification state. TanStack React Query manages cacheable server data such as assessment history and dashboard information.

Code 6.1 Centralized Frontend Route Mapping

```
01 export const PAGE_ROUTES: Record<Page, string> = {
02   landing: '/',
03   dashboard: '/dashboard',
04   assessments: '/assessments',
05   chatbot: '/chatbot',
06   journal: '/journal',
07   progress: '/progress',
08   admin: '/admin',
09   'therapist-portal': '/therapist_portal'
10 };
```

6.3 Backend API Implementation

The backend middleware chain applies security headers, compression, logging, health monitoring, timeouts, request tracking, CORS, sessions, Passport, JWT refresh, CSRF protection, parsing, sanitization, and no-store API headers before routing requests. Each functional area is mounted under /api and delegates to controllers and services.

Code 6.2 Representative API Route Registration

```
01 app.use('/api/auth', authRoutes);
02 app.use('/api/assessments', assessmentRoutes);
03 app.use('/api/chat', chatRoutes);
04 app.use('/api/conversations', conversationRoutes);
05 app.use('/api/mood', moodRoutes);
06 app.use('/api/journal', journalRoutes);
07 app.use('/api/content', contentRoutes);
08 app.use('/api/privacy', privacyRoutes);
09 app.use('/api/admin', adminRoutes);
10 app.use('/api/therapist-portal', therapistPortalRoutes);
```

Table 6.2 Core API Groups

API Group	Purpose
/api/auth	Registration, login, OAuth, current-user and password workflows.
/api/assessments	Assessment definitions, sessions, scoring, history, and insights.
/api/chat and /api/conversations	Messages, conversation history, feedback, memory, and export.
/api/mood, /journal, /sleep, /habits	Daily wellbeing tracking and reflection.
/api/content and /practices	Resource catalogue, engagement, bookmarks, and recommendations.
/api/crisis and /support	Crisis resources, safety plans, FAQ, and tickets.
/api/admin	Administrative management and analytics.
/api/therapist-portal	Therapist authentication, bookings, client context, and notes.
/api/privacy	Consent, data export, and account deletion.

6.4 Authentication and Authorization

Code 6.3 JWT Authentication Middleware

```
01 const authHeader = req.headers.authorization;
02 if (!authHeader?.startsWith('Bearer ')) {
03   res.status(401).json({ success: false, error: 'Access denied.' });
04   return;
05 }
06 const token = authHeader.substring(7);
07 const decoded = jwt.verify(token, getJwtSecret()) as JwtPayload;
08 const user = await prisma.user.findUnique({ where: { id: decoded.id } });
09 (req as AuthRequest).user = user;
10 next();
```

6.5 Database Implementation

Prisma supplies generated TypeScript types and a consistent data API. A singleton client avoids unnecessary connection pools during development. The bootstrap process normalizes local file URLs, validates environment variables, generates the correct Prisma client, synchronizes the local schema, and seeds demo and crisis-resource data.

Code 6.4 Prisma Singleton and Graceful Disconnect

```
01 export const prisma =
02   globalThis.prismaGlobal ?? prismaClientSingleton();
03
04 if (process.env.NODE_ENV !== 'production') {
05   globalThis.prismaGlobal = prisma;
06 }
07
08 process.on('beforeExit', async () => {
09   await prisma.$disconnect();
10 });
```

6.6 Multi-Provider AI Service

Provider adapters implement a shared AI interface. Providers are initialized only when valid credentials are available. Priority is configurable through environment variables. Repeated failures place a provider into cooldown, availability is cached, and the last working provider can be preferred to reduce latency.

Code 6.5 AI Provider Adapter Initialization

```
01 switch (type as AIProviderType) {  
02     case 'openai': provider = new OpenAIProvider(config); break;  
03     case 'anthropic': provider = new AnthropicProvider(config); break;  
04     case 'gemini': provider = new GeminiProvider(config); break;  
05     case 'nvidia': provider = new NvidiaProvider(config); break;  
06     case 'huggingface': provider = new HuggingFaceProvider(config); break;  
07     case 'ollama': provider = new OllamaProvider(config); break;  
08 }
```

6.7 Assessment and Insight Processing

Assessment definitions and scoring are separated from UI rendering. Results store raw, maximum, normalized, and category scores. Combined assessment sessions allow multiple short screens to be completed in one guided flow. Insight services calculate current wellness scores, category summaries, trends, and AI-assisted explanatory text while preserving the original results.

6.8 Personalization and Recommendations

Recommendation services combine assessment results, mood, content engagement, plans, and profile preferences. Content categories include meditation, breathing, mindfulness, journaling, CBT techniques, grounding, movement, yoga, sleep hygiene, articles, audio, video, stories, and psychoeducation.

CHAPTER 7

SECURITY, PRIVACY, SAFETY AND ETHICS

CHAPTER 7

SECURITY, PRIVACY, SAFETY AND ETHICS

7.1 Security Controls

- Helmet security headers and production HSTS.
- CORS origin validation and credential-aware requests.
- JWT authentication with automatic refresh support.
- Passport and Google OAuth integration.
- CSRF protection for state-changing requests.
- Input sanitization, schema validation, and Prisma parameterization.
- Authentication-specific and production rate limits.
- Request timeouts, logging, readiness checks, and graceful shutdown.

7.2 Privacy and Data Control

The User model contains consent and sharing preferences such as data consent, clinician sharing, anonymous analytics, marketing communication, and research participation. Privacy APIs allow users to review or update these controls, export their records, and request account deletion. Sensitive keys and provider credentials are loaded from environment variables and are not committed to source control.

7.3 Crisis Detection

Crisis analysis is multi-layered. Recent user messages are evaluated for severity-weighted patterns. Assessment results and mood trajectories provide longitudinal context. Engagement changes can provide a weaker supplementary signal. AI-assisted classification is constrained by a safety-specific prompt and does not replace deterministic rules. High and critical states trigger immediate-action behavior and analytics events.

Code 7.1 Severity-Weighted Crisis Pattern Examples

```
01 const CRISIS_PATTERNS = {  
02   CRITICAL: [  
03     /\b(suicide|suicidal|kill myself|end (my|it all))\b/i,  
04     /\b(want to die|better off dead|no reason to live)\b/i,  
05     /\b(self[ -]?harm|cutting|overdose)\b/i  
06   ],  
07   HIGH: [  
08     /\b(hopeless|worthless|no point)\b/i,  
09     /\b(hurt myself|harm myself)\b/i  
10   ]  
11 };
```

7.4 Ethical Boundaries

- MaanSarathi is described as a wellbeing platform, not a clinician or diagnostic system.
- AI responses must avoid presenting diagnoses, prescriptions, or guaranteed outcomes.
- High-risk situations should direct users toward emergency or professional support.
- Users should know when they are interacting with AI and how their data is used.
- Assessment results should be presented as screening insights rather than medical conclusions.
- Human review and professional governance are necessary before large-scale clinical deployment.

CHAPTER 8

TESTING AND VALIDATION

CHAPTER 8

TESTING AND VALIDATION

8.1 Testing Strategy

Table 8.1 Testing Strategy

Test Level	Tools/Method	Focus
Static Verification	TypeScript compiler and ESLint	Types, contracts, imports, unsafe patterns.
Unit Testing	Vitest	Scoring, utilities, provider fallback, crisis logic.
API Integration	Vitest and Supertest	Routes, middleware, database behavior, response contracts.
Frontend Component	Testing Library and jsdom	Rendering, interactions, loading/error states.
Build Verification	Vite and TypeScript builds	Production compilation and dependency integration.
Manual Workflow	Browser-based end-to-end review	Authentication, onboarding, dashboard, chat, admin, therapist.
Security Review	Configuration and middleware inspection	CORS, tokens, CSRF, secrets, validation, rate limiting.

8.2 Representative Test Cases

Table 8.2 Representative Test Cases

ID	Scenario	Expected Result
TC-01	Register with valid user details.	Account created; verification/onboarding flow begins.
TC-02	Access protected API without token.	401 response; no protected data returned.
TC-03	Complete a standardized assessment.	Scores stored; history and insight data updated.
TC-04	Send a normal wellbeing message.	Contextual AI response stored in conversation history.
TC-05	Primary AI provider fails.	Next available provider is attempted according to priority.
TC-06	Message contains explicit crisis indicators.	High/critical result, emergency resources, immediate-action path.
TC-07	Log mood and journal entries.	Records stored and reflected in dashboard/progress data.

ID	Scenario	Expected Result
TC-08	Admin accesses protected management route.	Authorized admin receives data; ordinary user is rejected.
TC-09	Therapist saves a session note.	Note stored for permitted therapist/client relationship.
TC-10	User exports personal data.	Structured export generated for the authenticated user.
TC-11	Readiness endpoint with unavailable database.	503 degraded response reports database failure.
TC-12	Frontend production build.	Vite outputs optimized distributable files without type failure.

8.3 Validation Considerations

Mental-health applications require more than technical correctness. Future validation should include usability studies, accessibility audits, cultural review, clinician review of assessment interpretations and safety messages, privacy impact assessment, bias evaluation, and controlled longitudinal evaluation. Crisis-detection sensitivity and specificity must be evaluated carefully to reduce both missed risk and unnecessary escalation.

CHAPTER 9

RESULTS AND DISCUSSION

CHAPTER 9

RESULTS AND DISCUSSION

9.1 Implemented User Experience

The completed web application provides a public landing experience, account creation and login, onboarding, a personalized dashboard, assessments, insights, plans, AI companion chat, content library, practices, journal, games, progress, profile, and help/safety screens. The interface is responsive and uses shared UI primitives for consistent interaction.

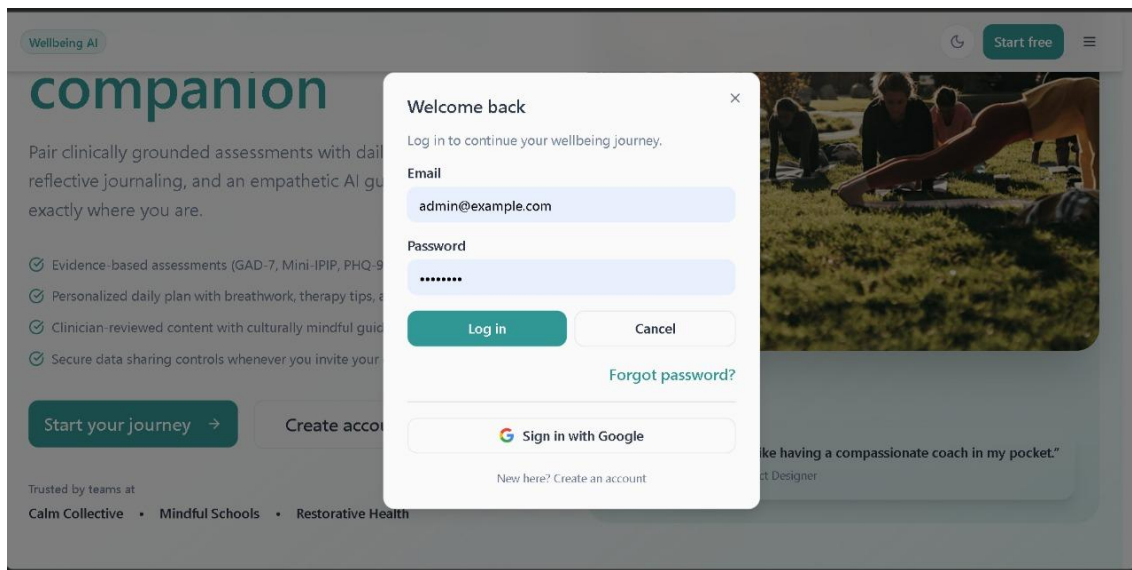


Figure 9.1 User Authentication Screen

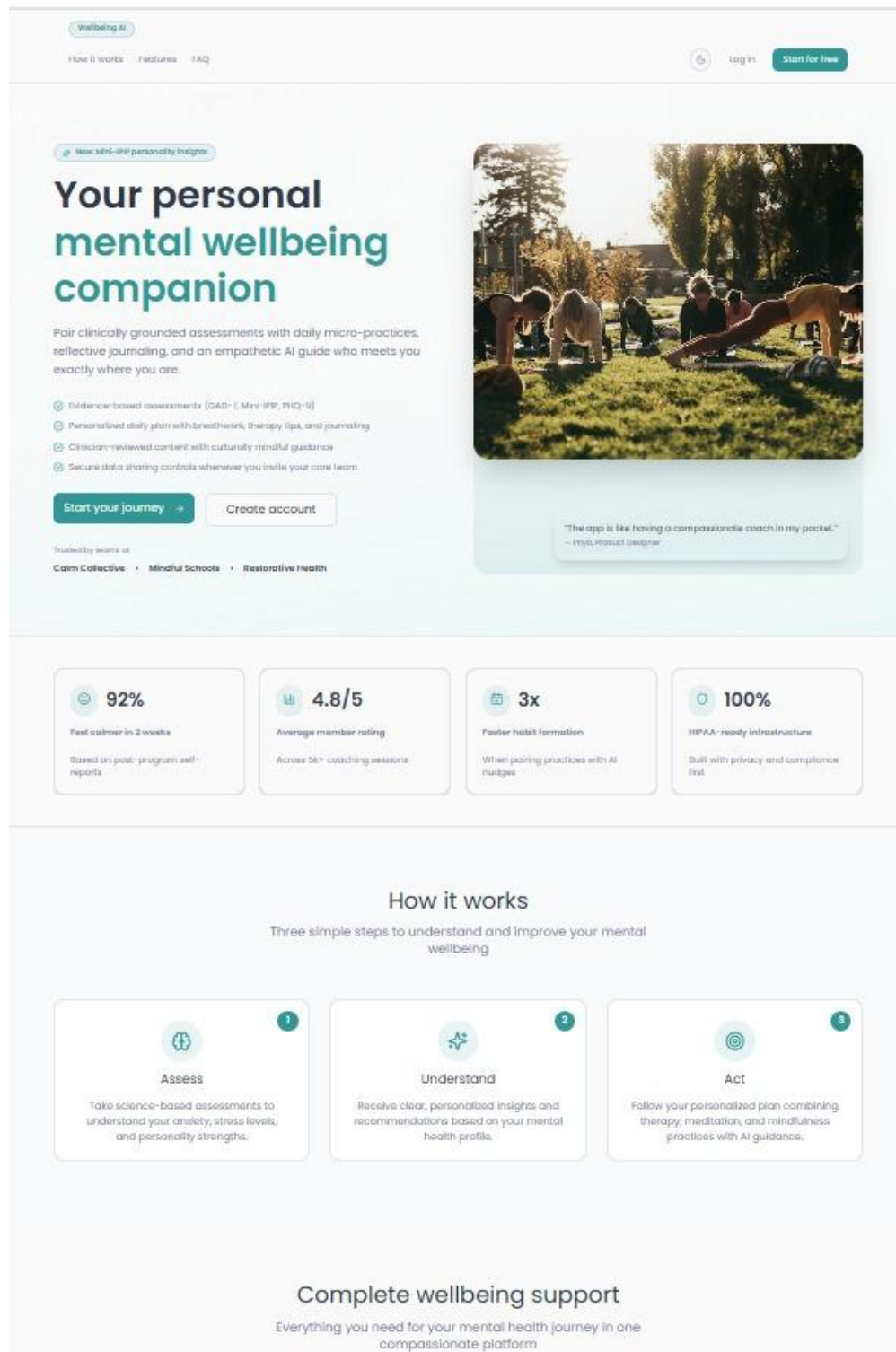


Figure 9.2 Public Landing Page

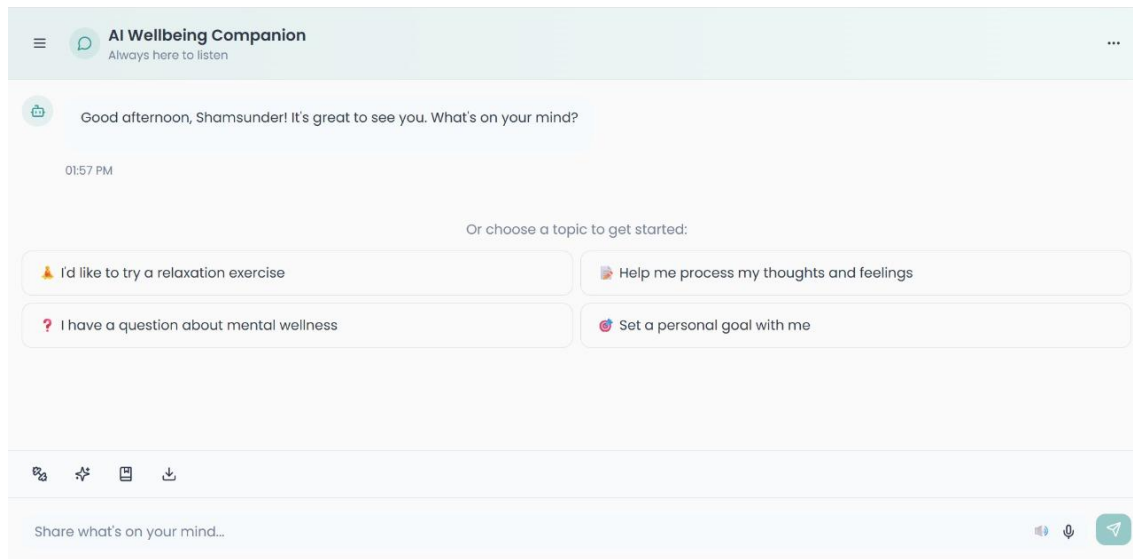


Figure 9.3 AI Companion Interface

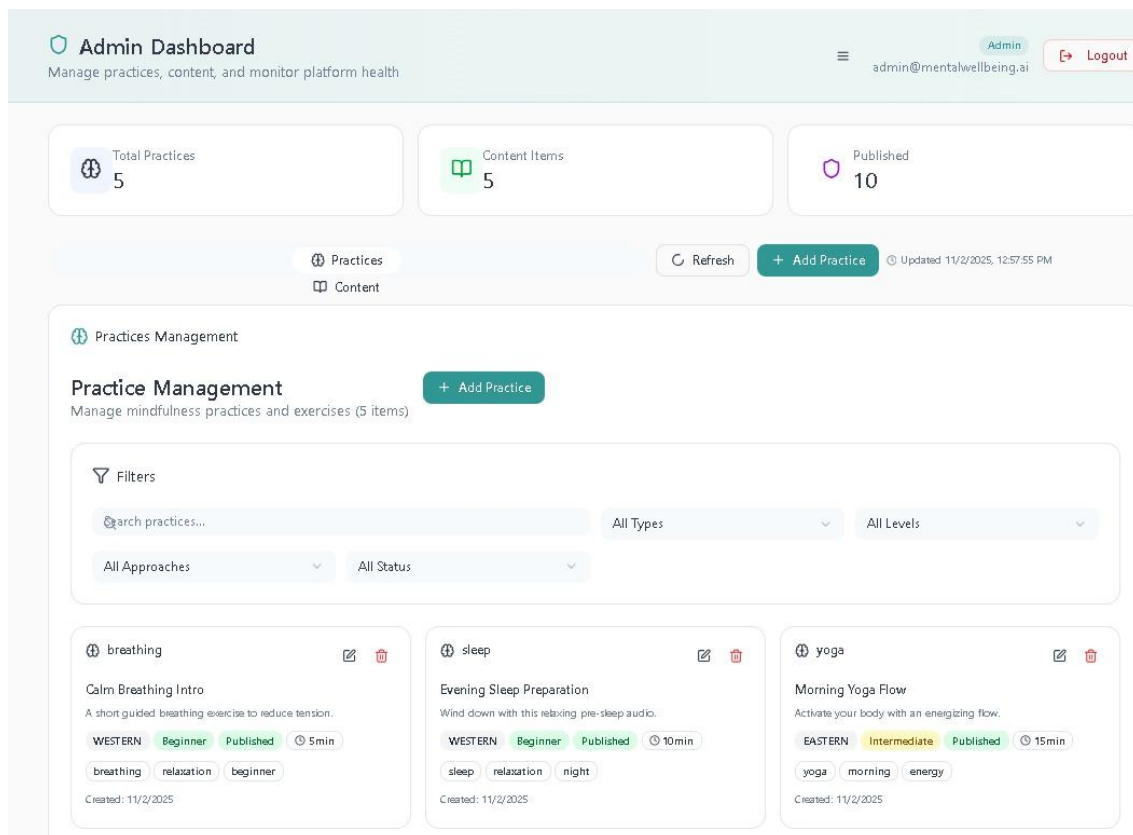


Figure 9.4 Admin Dashboard

9.2 Result Summary

[← Back](#)

Step 5 of 7

Choose Your Wellbeing Approach

Select the approach that resonates most with you. You can change this anytime.



Therapy-Based (Western)

Focus on evidence-based therapy techniques, cognitive behavioral strategies, and modern psychological approaches.

CBT & DBT techniquesStructured assessmentsGoal-oriented plansProgress tracking



Yoga-Based (Eastern)

Emphasize mindfulness, meditation, yoga practices, and ancient wisdom for holistic wellbeing.

Meditation practicesYoga sessionsBreathworkMind-body connection



Hybrid (Best of Both)

Combine Western therapy techniques with Eastern practices for a comprehensive, integrated approach.

Balanced approachDiverse techniquesHolistic healingFlexible methods

Figure 9.4 Onboarding approach section1

Page | 55

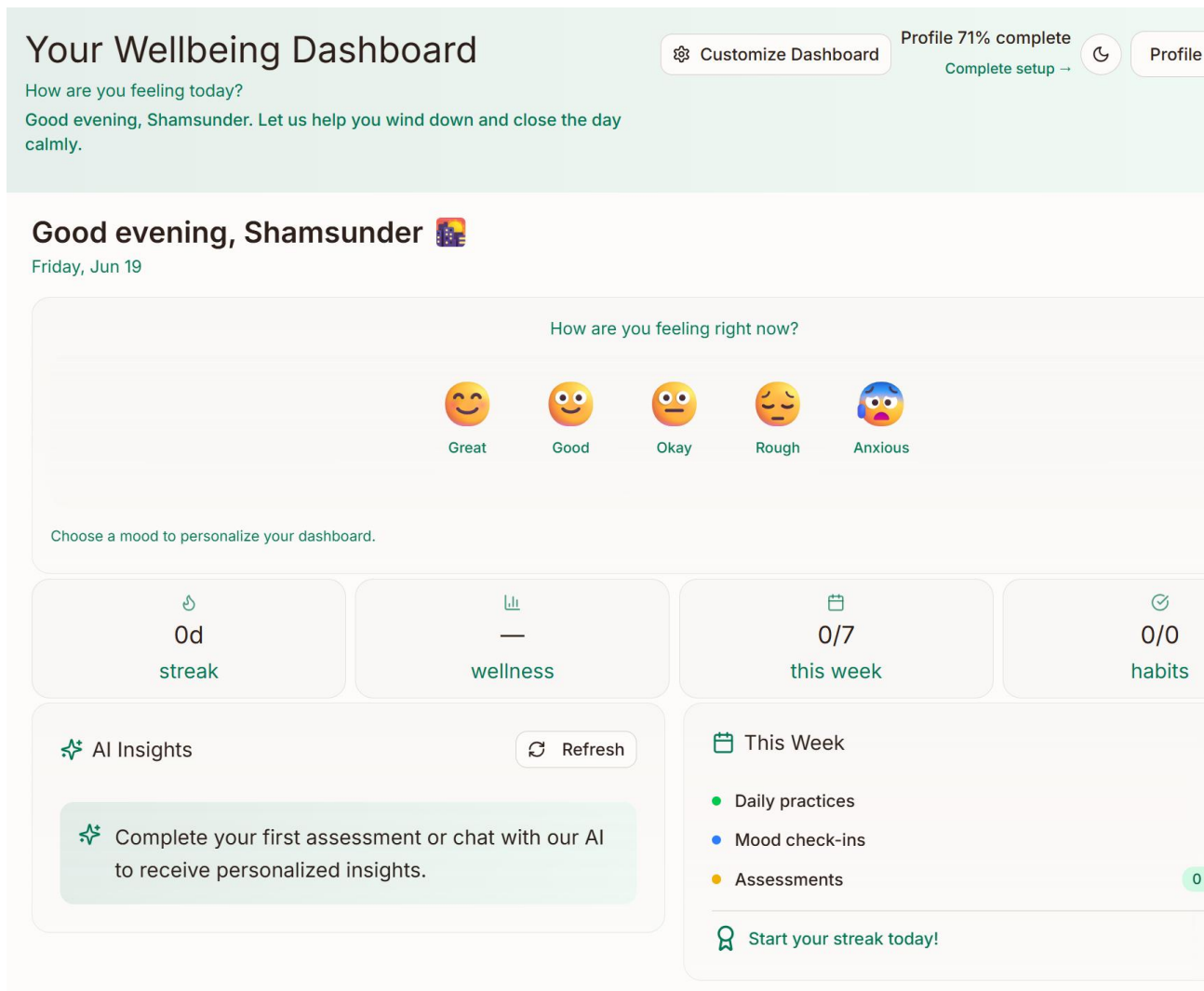
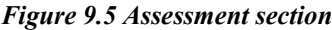


Figure 9.5 Dashboard1



[← Back to Assessments](#)
Session Standalone

GAD-7 (Anxiety Assessment)
 Generalized Anxiety Disorder 7-item assessment.

Question 2 of 7

~3 min remaining

29% complete

Not being able to stop or control worrying

☐ Not at all

☒ Several days

☐ More than half the days

☐ Nearly every day

[← Previous](#)

[Next →](#)

Figure 9.6 Assessment Flow

Figure 9.7 Chatbot 1

Page | 58

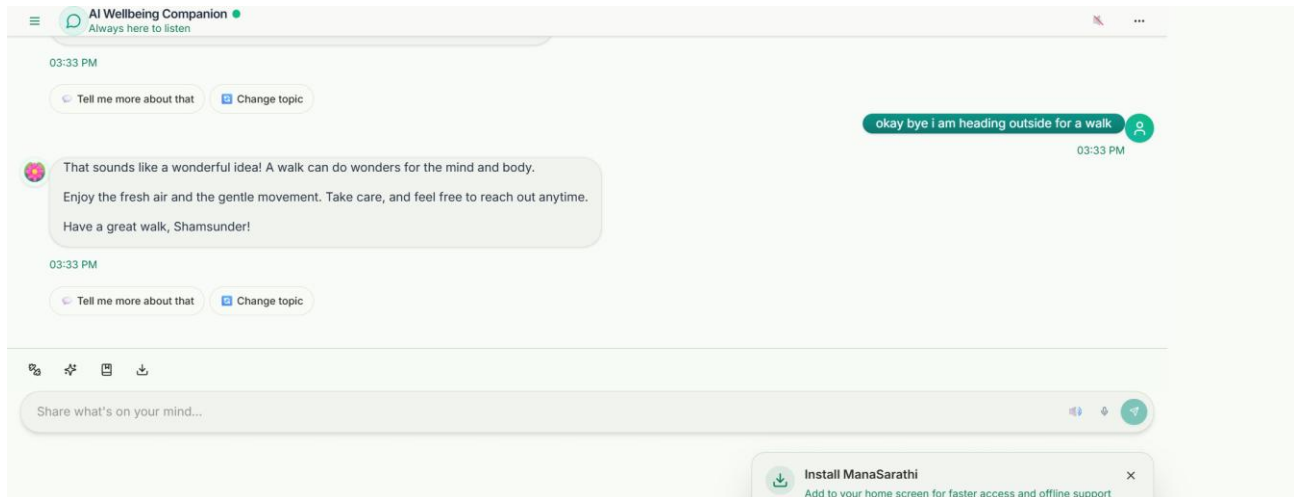


Figure 9.8 Chatbot 2

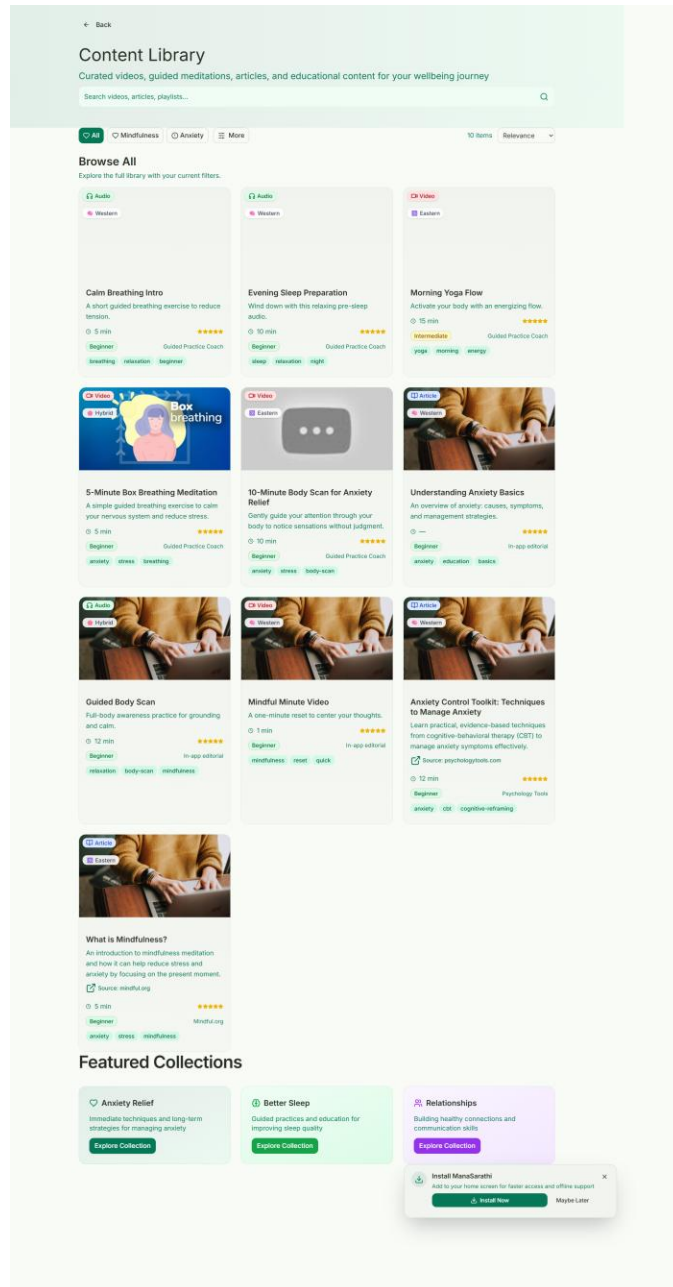


Figure 9.8 Content Library

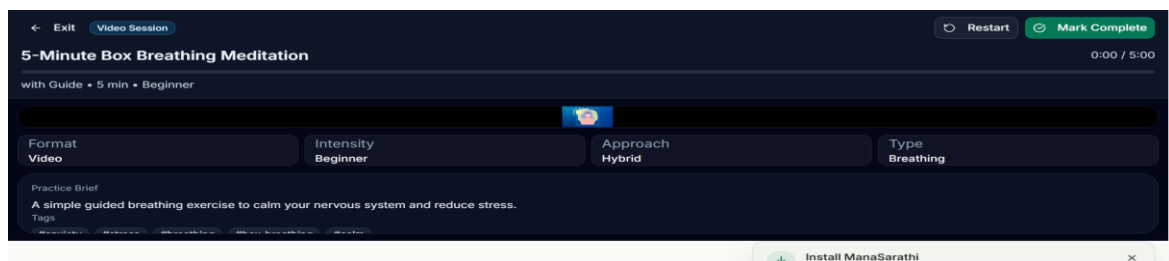


Figure 9.10 Practice session

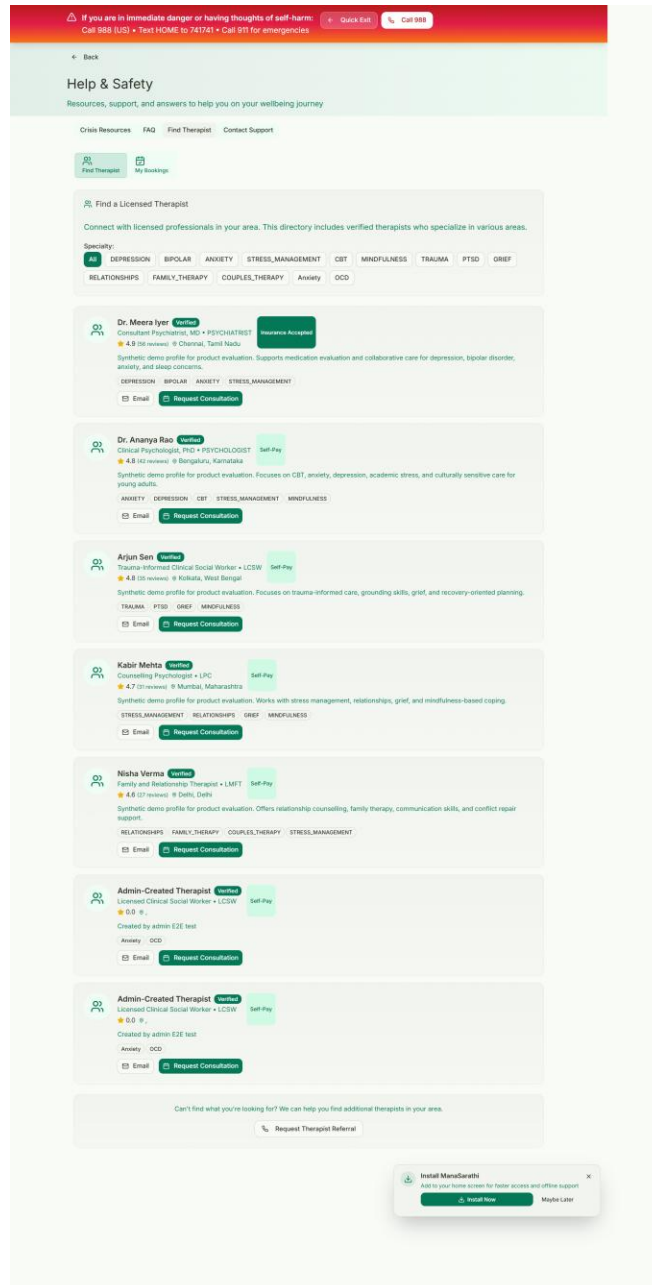


Figure 10.1 Help Section / Therapist Booking & Overview

Table 9.1 Result Summary

Area	Observed Result
Architecture	Frontend and backend are separated into npm workspaces with clear module boundaries.
Authentication	Email/password, JWT, Google OAuth, onboarding, and protected routes are implemented.

Area	Observed Result
Assessments	Individual/combined sessions, scoring, history, and insights are supported.
AI Companion	Context, memory, provider fallback, crisis checks, history, and feedback are integrated.
Tracking	Mood, sleep, journal, gratitude, intentions, habits, and check-ins are persisted.
Content	Practices, media, engagement, bookmarks, and recommendations are available.
Professional Oversight	Admin and therapist portals provide management and review workflows.
Privacy/Safety	Consent, export, deletion, crisis resources, safety plans, FAQ, and support tickets are present.
Deployment	Local setup automation and production configurations are included.

9.3 Discussion

The project demonstrates the value of integrating multiple wellness workflows around a shared user model. Conversation memory and longitudinal records improve personalization, while separate administrative and therapist portals create governance paths that are absent in many standalone chatbots. The provider abstraction reduces dependency on one model vendor. The main limitations are the need for stronger clinical validation, production-scale monitoring, broader automated test coverage, and careful review of generated AI content.

CHAPTER 10

DEPLOYMENT, OPERATION AND MAINTENANCE

CHAPTER 10

DEPLOYMENT, OPERATION AND MAINTENANCE

10.1 Local Setup

1. Install Node.js 18 or later and npm 8 or later.
2. Run `npm run setup:local` to install dependencies, create local environment files, generate Prisma, synchronize SQLite, and seed local data.
3. Run `npm run doctor:config` to validate required configuration.
4. Run `npm run dev` to start the backend and then the frontend.
5. Open the frontend at `http://localhost:3000` and verify the API at `http://localhost:5000/api/health`.

10.2 Production Architecture

The repository includes Vercel configuration for the frontend and Render configuration for backend/database deployment. In production, the frontend is built with `VITE_API_URL` pointing to the backend API. The backend uses PostgreSQL, secure JWT/session secrets, configured OAuth credentials, approved CORS origins, HTTPS cookies, and production rate limiting.

10.3 Health and Monitoring

The `/api/health` endpoint confirms process availability. The `/api/health/ready` endpoint verifies database connectivity and AI provider availability and returns a degraded status when dependencies are unavailable. Pino structured logs, request IDs, system-health middleware, slow-query logging, and graceful shutdown support diagnosis and maintenance.

10.4 Backup and Recovery

- Use managed PostgreSQL backups and periodically verify restoration.
- Store uploaded media in durable storage for production use.
- Keep environment secrets in deployment secret managers.
- Apply Prisma migrations before application promotion.
- Maintain audit and activity logs according to approved retention policies.
- Test provider fallback and database readiness after deployment changes.

CHAPTER 11

CONCLUSION AND FUTURE SCOPE

CHAPTER 11

CONCLUSION AND FUTURE SCOPE

11.1 Conclusion

MaanSarathi delivers a comprehensive foundation for digital mental-wellness support. The project integrates structured assessment, daily reflection, personalized content, progress monitoring, AI-assisted conversation, crisis-aware handling, privacy controls, and role-based professional oversight in one modular full-stack system. The architecture is maintainable and deployable, and the implementation demonstrates practical integration of modern web technologies, relational data modelling, and multiple AI providers.

The project also demonstrates that safety and privacy must be architectural concerns. Crisis detection, consent, data control, provider abstraction, logging, and health checks are embedded across the system rather than treated as isolated features. With appropriate clinical governance and validation, the platform can evolve into a useful companion for self-care and professional support.

11.2 Limitations

- AI responses can still be incorrect, incomplete, biased, or culturally inappropriate.
- Crisis detection has not been clinically validated for diagnostic or emergency use.
- The current repository primarily delivers the web frontend and backend workspaces.
- Large-scale performance, security, and longitudinal outcome studies remain pending.
- Some features depend on third-party providers and internet availability.
- Professional workflows require organizational policy, consent, and access governance.

11.3 Future Scope

- Native Android and iOS application with encrypted offline support and notifications.
- Clinician-reviewed assessment catalogue and configurable regional scoring guidance.
- Telehealth integration, verified therapist directory, and consent-controlled sharing.
- Improved multilingual and culturally adaptive AI responses.
- Wearable integration for sleep, activity, and stress-related signals with explicit consent.
- Federated or privacy-preserving analytics for population insights.
- Human-in-the-loop review tools for high-risk AI conversations.
- Formal accessibility certification, penetration testing, and privacy impact assessment.
- Controlled clinical and university-based evaluation of usability and wellbeing outcomes.

- More extensive automated browser, API, load, and provider-resilience testing.

REFERENCES

REFERENCES

- [1] A. Thakkar, A. Gupta, and A. De Sousa, “Artificial Intelligence in Positive Mental Health: A Narrative Review,” *Frontiers in Digital Health*, 2024.
- [2] Z. B. V. Salcedo et al., “Artificial Intelligence and Mental Health Issues: A Narrative Review,” *Journal of Public Health Sciences*, 2023.
- [3] J. A. Ruiz-Vanoye et al., “Artificial Intelligence and Human Well-Being: A Review of Applications and Effects on Life Satisfaction,” 2025.
- [4] L. Laranjo et al., “The Effects of an AI-Powered Chatbot on Mental Health,” 2023.
- [5] Y. Guo et al., “Large Language Models for Mental Health Support: A Systematic Review,” 2024.
- [6] R. A. Calvo et al., “Natural Language Processing in Mental Health Applications Using Non-Clinical Texts,” 2025.
- [7] S. Poria et al., “A Survey on Sentiment Analysis and Emotion Recognition,” 2023.
- [8] S. Halder, “Developing Mental Health Support Chatbots in India: Challenges and Insights,” *Annals of Indian Psychiatry*, 2025.
- [9] A. Marade et al., “Spiritual.AI: An AI-Driven Platform for Enhancing Mental, Emotional, and Spiritual Well-Being,” *IJRASET*, 2025.
- [10] M. Casu et al., “AI Chatbots for Mental Health: A Scoping Review of Effectiveness, Feasibility, and Applications,” *Applied Sciences*, 2024.
- [11] K. K. Fitzpatrick, A. Darcy, and M. Vierhile, “Delivering Cognitive Behavior Therapy to Young Adults Using a Fully Automated Conversational Agent,” *JMIR Mental Health*, 2017.
- [12] Express.js Documentation, <https://expressjs.com/>.
- [13] React Documentation, <https://react.dev/>.
- [14] Prisma ORM Documentation, <https://www.prisma.io/docs/>.
- [15] Vite Documentation, <https://vite.dev/>.
- [16] OWASP Foundation, “OWASP Top Ten Web Application Security Risks,” <https://owasp.org/>.
- [17] World Health Organization, “Mental Health: Strengthening Our Response,” <https://www.who.int/>.

APPENDIX A

APPENDIX A

PROJECT DIRECTORY STRUCTURE

Appendix A.1 Repository Structure

```
01 MaanSarathi/  
02 |-- frontend/  
03 |   |-- src/  
04 |       |-- admin/  
05 |       |-- therapist/  
06 |       |-- components/features/  
07 |       |-- contexts/  
08 |       |-- hooks/  
09 |       |-- services/  
10 |       |-- stores/  
11 |       |-- App.tsx  
12 |-- backend/  
13 |   |-- src/  
14 |       |-- controllers/  
15 |       |-- routes/  
16 |       |-- services/providers/  
17 |       |-- middleware/  
18 |       |-- server.ts  
19 |   |-- prisma/  
20 |       |-- schema.prisma  
21 |       |-- schema.local.prisma  
22 |-- shared/  
23 |-- scripts/  
24 |-- package.json
```

APPENDIX B

APPENDIX B

ENVIRONMENT CONFIGURATION REFERENCE

Appendix B.1 Example Environment Variables

```
01 # Backend
02 NODE_ENV=development
03 PORT=5000
04 DATABASE_URL=file:./prisma/dev.db
05 JWT_SECRET=<secure-random-secret>
06 SESSION_SECRET=<secure-random-secret>
07 FRONTEND_URL=http://localhost:3000
08
09 # AI provider configuration
10 AI_PROVIDER_PRIORITY=gemini,openai,anthropic,nvidia,huggingface,ollama
11 AI_ENABLE_FALLBACK=true
12 GEMINI_API_KEY_1=<secret>
13
14 # Frontend
15 VITE_API_URL=http://localhost:5000/api
```

APPENDIX C

APPENDIX C

IMPORTANT OPERATING COMMANDS

Command	Purpose
npm run setup:local	Install workspaces, prepare environment, generate/sync database, and seed local data.
npm run doctor:config	Validate configuration and required environment variables.
npm run dev	Start backend and frontend development servers.
npm run build	Build frontend and backend for production.
npm run typecheck	Run TypeScript checks in both workspaces.
npm run test	Run backend and frontend test suites.
npm run ci	Run lint, typecheck, build, and tests.
npm run db:studio	Open Prisma Studio for database inspection.